INSTITUTO UNIVERSITÁRIO DE LISBOA

Mestrado em Engenharia de Telecomunicações e Informática
Segurança em Redes e Sistemas de Informação

Deteção e Contenção Automática de Dispositivos IoT Comprometidos com *Machine Learning*: Uma Revisão Sistemática da Literatura

Afonso Duarte Rosado Lopes (122631), Rafael Fernandes dos Reis (122616), Rafael Andrade da Silva (122638), Gonçalo José Bilé Batista (122623), João Azinheira Alves (123488)
Iscte – Instituto Universitário de Lisboa
Docentes: Prof. Valderi Reis Quietinho Leithardt e Prof. Maurício Machado da Rosa
Data: 29/10/2026

***Resumo*— Os dispositivos da Internet das Coisas (IoT) têm poucos recursos, raramente são atualizados e são facilmente comprometidos por *malware* que os recruta para *botnets*, como a Mirai. Quando um deles é comprometido, continua a funcionar normalmente aos olhos do utilizador, e o tráfego de rede é muitas vezes a única forma de o detetar. Este trabalho apresenta uma revisão sistemática da literatura sobre a deteção de dispositivos IoT comprometidos com *machine learning* a partir do tráfego de rede, com foco no passo seguinte à deteção: a resposta automática que contém o dispositivo. Seguindo o protocolo da unidade curricular, foram pesquisadas cinco bases de dados (677 resultados). Os estudos foram selecionados com critérios de inclusão e exclusão, um critério específico sobre a existência de resposta automática e uma avaliação de qualidade, resultando em 31 estudos incluídos. A análise mostra que muitos trabalhos anunciam mecanismos de mitigação mas poucos os implementam e avaliam, e que a avaliação em protótipo ou em tempo real continua a ser minoritária. Com base nestas conclusões, propõe-se uma arquitetura para um *gateway* que extrai fluxos com Zeek, compara modelos supervisionados e não supervisionados e coloca automaticamente em quarentena os dispositivos suspeitos, enquadrada numa arquitetura *Zero Trust* segundo o NIST SP 800-207.**

***Palavras-chave*—Internet das Coisas, deteção de intrusões, *machine learning*, *botnets*, contenção automática, revisão sistemática**

ÍNDICE

Resumo 1
Glossário 1
I Introdução 2
II Fundamentos 3
III Metodologia da revisão sistemática 5
IV Resultados da revisão 8
V Solução proposta 17
VI Enquadramento na arquitetura Zero Trust 19
VII Discussão 22
VIII Conclusões e trabalhos futuros 23
Referências 24

GLOSSÁRIO

ACL	Lista de controlo de acessos (*Access Control List*)
AE	*Autoencoder*
AME	Eficácia de mitigação de ataques (*Attack Mitigation Effectiveness*)
AUC, AUROC	Área sob a curva ROC (*Area Under the ROC Curve*)
AVAC	Aquecimento, ventilação e ar condicionado
C&C	Comando e controlo (*Command and Control*)
CIL, DIL	*Class Incremental Learning*, *Domain Incremental Learning*
CISA	*Cybersecurity and Infrastructure Security Agency*
CNN	Rede neuronal convolucional (*Convolutional Neural Network*)
CoAP	*Constrained Application Protocol*
CPS	Sistema ciber-físico (*Cyber-Physical System*)
DDoS	Negação de serviço distribuída (*Distributed Denial of Service*)
DL	Aprendizagem profunda (*Deep Learning*)
DNS	*Domain Name System*
DoS	Negação de serviço (*Denial of Service*)
DT	Árvore de decisão (*Decision Tree*)
EC	Critério de exclusão (*Exclusion Criterion*)
FAR	Taxa de falsos alarmes (*False Alarm Rate*)
FN, FP	Falsos negativos, falsos positivos
FNR	Taxa de falsos negativos (*False Negative Rate*)
FPR	Taxa de falsos positivos (*False Positive Rate*)
GNN	Rede neuronal de grafos (*Graph Neural Network*)
GRU, BiGRU	*Gated Recurrent Unit* (bidirecional)
IC	Critério de inclusão (*Inclusion Criterion*)
IDS	Sistema de deteção de intrusões (*Intrusion Detection System*)
IF	*Isolation Forest*
IoT	Internet das Coisas (*Internet of Things*)
IP	*Internet Protocol*
IPS, IRS	Sistema de prevenção (*Intrusion Prevention System*) e de resposta a intrusões (*Intrusion Response System*)
IPFIX	*IP Flow Information Export*
KNN	*k-Nearest Neighbors*
LR	Regressão logística (*Logistic Regression*)
LSTM	*Long Short-Term Memory*
MAC	*Media Access Control*
MCC	*Matthews Correlation Coefficient*
MFA	Autenticação multifator (*Multi-Factor Authentication*)
MITM	*Man-in-the-Middle*
ML	Aprendizagem automática (*Machine Learning*)
MLP	*Multilayer Perceptron*
MQTT	*Message Queuing Telemetry Transport*

MUD	*Manufacturer Usage Description*
NB	Naive Bayes
NIST	*National Institute of Standards and Technology*
P4	*Programming Protocol-independent Packet Processors*
PA	*Policy Administrator*
PAM	*Polygon Area Metric*
PDP	*Policy Decision Point*
PDR	Taxa de entrega de pacotes (*Packet Delivery Ratio*)
PE	*Policy Engine*
PEP	*Policy Enforcement Point*
PIP	*Policy Information Point*
PIPOC	População, Intervenção, Comparação, Resultados (*Outcomes*), Contexto
QA	Avaliação de qualidade (*Quality Assessment*)
QE, QF, QG	Questão estatística, questão focada, questão geral
RF	*Random Forest*
RGPD	Regulamento Geral sobre a Proteção de Dados
RL	Aprendizagem por reforço (*Reinforcement Learning*)
RNN	Rede neuronal recorrente (*Recurrent Neural Network*)
RPL	*Routing Protocol for Low-Power and Lossy Networks*
RSL	Revisão sistemática da literatura
SDN	Redes definidas por *software* (*Software-Defined Networking*)
SP	*Special Publication* (série de documentos do NIST)
SQLi	Injeção de SQL (*SQL Injection*)
SRSI	Segurança em Redes e Sistemas de Informação
SVM	*Support Vector Machine*
TCP, UDP	*Transmission Control Protocol*; *User Datagram Protocol*
TN, TP	Verdadeiros negativos, verdadeiros positivos
TPR	Taxa de verdadeiros positivos (*True Positive Rate*), igual ao *recall*
VLAN	*Virtual Local Area Network*
WSN	Rede de sensores sem fios (*Wireless Sensor Network*)
XAI	Inteligência artificial explicável (*eXplainable Artificial Intelligence*)
XGBoost	*Extreme Gradient Boosting*
XSS	*Cross-Site Scripting*
ZTA	Arquitetura *Zero Trust* (*Zero Trust Architecture*)
ZTMM	*Zero Trust Maturity Model* (CISA)

I. Introdução

A Internet das Coisas (IoT) deixou de ser um conceito de nicho e passou a fazer parte de casas, hospitais, fábricas e cidades. Câmaras de vigilância, termóstatos, fechaduras, sensores e controladores industriais estão hoje ligados à rede de forma permanente e, em muitos casos, diretamente expostos à Internet. A maioria destes equipamentos foi pensada para ser barata e fácil de instalar, e não para resistir a ataques. Têm pouca memória e pouca capacidade de processamento, raramente recebem atualizações de *firmware* e não têm margem para correr um antivírus ou um agente de segurança. Muitos continuam a usar as credenciais de fábrica durante toda a vida útil.

A *botnet* Mirai mostrou em 2016 o que acontece quando estas fragilidades são exploradas em grande escala. O *malware* percorria a Internet à procura de dispositivos com o serviço Telnet acessível e tentava entrar com uma pequena lista de credenciais por omissão. No pico da sua atividade controlava cerca de 600 mil equipamentos, que foram usados em ataques de negação de serviço distribuída (DDoS) contra alvos como o fornecedor de DNS Dyn [1]. O código-fonte acabou por ser publicado e deu origem a inúmeras variantes, mas o padrão mantém-se: aos olhos do dono, o dispositivo infetado continua a funcionar normalmente, enquanto em segundo plano faz varrimentos, participa em ataques ou comunica com um servidor de comando e controlo (C&C).

Como não é possível instalar *software* de proteção nestes dispositivos, quem gere a rede depende quase só do tráfego para perceber que algo mudou. Esse tráfego pode ser observado num ponto central, como o *gateway*, sem tocar nos equipamentos. É neste cenário que a aprendizagem automática (ML) tem sido mais explorada. A partir de estatísticas de fluxos, como a duração, o volume, os intervalos entre pacotes ou as portas contactadas, os modelos aprendem a separar o comportamento habitual de cada equipamento dos padrões típicos de uma infeção. A literatura sobre esta forma de deteção é extensa e reporta com frequência taxas de acerto acima de 99% em *datasets* públicos, como se vê em vários dos estudos incluídos nesta revisão [2]–[6].

Detetar o problema, porém, é só metade do trabalho. Um alerta que fica à espera de ser lido deixa o dispositivo ativo na rede durante minutos ou horas, tempo suficiente para participar num ataque ou para infetar outros equipamentos. Numa casa ou numa pequena empresa, muitas vezes nem existe alguém para o ler. Faz por isso sentido que a deteção seja acompanhada de uma resposta automática, aplicada no mesmo ponto onde o tráfego é observado: bloquear o dispositivo, limitar o seu débito ou isolá-lo numa rede de quarentena. O trabalho de Noetzold *et al.* [7], que serve de ponto de partida a este projeto, integra modelos de *deep learning* num sistema de monitorização que deteta anomalias em vários domínios, entre eles processos suspeitos a partir de fluxos do CICIDS2017, e apresenta os alertas numa interface em tempo real. Entre as direções de trabalho futuro, os autores apontam a integração com sistemas de resposta a incidentes, a execução na *edge* e o uso de aprendizagem não supervisionada, e é precisamente esse passo que aqui se estuda.

Existem já várias revisões sobre segurança em IoT. Negera *et al.* [8] analisam a deteção de *botnets* com ML em redes IoT geridas por SDN; Kandah *et al.* [9] percorrem arquiteturas, ameaças emergentes e contramedidas adaptativas; Vera-Mundaca e Aquino-Trujillo [10] concentram-se nas ameaças e nas formas de mitigação em casas inteligentes. São visões abrangentes, mas nenhuma tem como objetivo central verificar se os mecanismos de contenção propostos chegam a ser implementados e avaliados, nem onde e como atuam. Esta revisão foca-se nesse ponto.

Objetivos. O trabalho tem dois objetivos. O primeiro é perceber, através de uma revisão sistemática da literatura (RSL), como é que a análise de tráfego com ML tem sido usada para detetar e conter automaticamente dispositivos IoT comprometidos. Interessa saber que modelos e *features* são usados, com que *datasets* e métricas são avaliados, que mecanismos de resposta são propostos e onde atuam, e se as soluções foram testadas para lá de um *dataset* estático. Estas questões estão formalizadas nas perguntas de pesquisa da Tabela III. O segundo objetivo é usar o resultado da revisão para desenhar a solução que será implementada na Parte 2 do projeto: um *gateway* que analisa o tráfego dos dispositivos e, ao detetar um comprometimento, os coloca automaticamente em quarentena.

Contribuições. As contribuições deste trabalho são quatro:

- uma RSL de 31 estudos publicados entre 2020 e 2026, selecionados a partir de 677 resultados em cinco bases de dados, com um critério explícito sobre a existência de resposta automática e uma avaliação de qualidade de cada estudo, que mostra que só 13 deles descrevem e avaliam a resposta e só oito medem o tempo que esta demora a conter um ataque;
- uma comparação dos trabalhos relacionados (Tabelas XIII e XIV) centrada na forma como a resposta é desencadeada, onde é aplicada e como é avaliada;
- a proposta de uma arquitetura de deteção e contenção para a Parte 2, assente na extração de fluxos com Zeek, em modelos supervisionados e não supervisionados e na quarentena com nftables, a avaliar com o *dataset* IoT-23 [11];
- o enquadramento dessa arquitetura no modelo *Zero Trust* do NIST SP 800-207 e no modelo de maturidade da CISA, aplicado a um edifício inteligente.

Estrutura. A Secção II apresenta os conceitos base. A Secção III descreve o protocolo da revisão sistemática. A Secção IV responde às perguntas de pesquisa e compara os trabalhos relacionados. A Secção V apresenta a solução proposta e a Secção VI enquadra-a numa arquitetura *Zero Trust*. A Secção VII discute as lacunas e as ameaças à validade e a Secção VIII conclui.

II. Fundamentos

A. Ameaças em dispositivos IoT

Um dispositivo IoT comprometido é ao mesmo tempo vítima e plataforma de ataque, e por isso não basta detetá-lo: é preciso contê-lo. Três características tornam estes dispositivos vulneráveis. A primeira são os recursos limitados: com pouca capacidade de processamento, pouca memória e pouca energia, não conseguem correr um antivírus nem cifra pesada, e a defesa tende a subir para o *gateway*, para o *edge* ou para o controlador SDN [12]. A segunda é a segurança fraca por omissão: credenciais por defeito, *firmware* raramente atualizado e protocolos leves, como o MQTT, o CoAP, o Zigbee ou o LoRaWAN, em que a autenticação e a cifra não são obrigatórias [13], [14]. Foi das credenciais por omissão que a Mirai se aproveitou para se propagar [1]. A terceira é a heterogeneidade e a escala: milhares de dispositivos de fabricantes e protocolos diferentes tornam difícil manter uma política de segurança uniforme.

A Tabela I resume as famílias de ataque consideradas nesta revisão e o sinal que cada uma deixa no tráfego. Um dispositivo comprometido altera o seu comportamento de rede: abre mais fluxos, contacta destinos novos, muda de ritmo. É este desvio, observável no tráfego, que os modelos de ML aprendem a identificar.

Tabela I
Famílias de ataque a dispositivos IoT consideradas na revisão

Família	O que é	Sinal no tráfego
DoS/DDoS	Inundação de um alvo (SYN, UDP, ICMP, HTTP) até esgotar a largura de banda ou os recursos	Pico de pacotes ou fluxos por segundo; muitas origens para um destino
Botnet e C&C	Dispositivos infetados que recebem ordens de um servidor de comando e controlo e propagam a infeção, como na Mirai [1]	Ligações periódicas a um destino externo; varrimento de portas (Telnet); tráfego de saída anómalo
Força bruta	Tentativas repetidas de *login* ou de autenticação	Muitas ligações curtas ao mesmo serviço
Malware e injeção	Código malicioso, injeção de comandos ou de dados, exploração de vulnerabilidades	*Payloads* e padrões de protocolo fora do normal
Reconhecimento	Varrimento de portas e *fingerprinting* que preparam o ataque	Muitos destinos ou portas, com poucos *bytes* por fluxo
Ataques de protocolo e de rede	MITM, *spoofing*, *black-hole*, *selective forwarding* (WSN), ataques ao MQTT [15]	Anomalias de endereçamento, encaminhamento ou temporização

B. Deteção de intrusões baseada em fluxos

A deteção por fluxos analisa o resumo estatístico de uma comunicação, e não o conteúdo dos pacotes, o que a torna leve e compatível com tráfego cifrado. Um fluxo é o conjunto de pacotes que partilham o mesmo 5-tuplo (endereços IP e portas de origem e de destino, e protocolo) dentro de uma janela de tempo ou até expirar um *timeout* [16]. Os fluxos podem ser exportados pelos próprios equipamentos de rede, com NetFlow ou IPFIX [16], [17], extraídos de capturas com ferramentas como o CICFlowMeter [18] ou o Zeek [19], ou lidos pelo controlador SDN a partir dos contadores dos *switches* OpenFlow.

O processo típico tem quatro passos: captar o tráfego no *gateway*, no *edge* ou no *switch*; agregá-lo em fluxos ou em janelas de tempo; calcular as *features*; e classificá-lo com um modelo de ML, que decide se o fluxo ou o dispositivo é benigno ou malicioso. A Tabela II apresenta os grupos de *features* mais comuns.

Tabela II
Grupos de features habitualmente extraídas de fluxos de rede

Grupo	Exemplos
Volume	Número de pacotes e de *bytes* por fluxo; pacotes e *bytes* por segundo
Temporais	Duração do fluxo; tempo entre pacotes (média, desvio-padrão)
Tamanho dos pacotes	Mínimo, máximo, média e desvio-padrão
Flags e protocolo	Contagem de *flags* TCP (SYN, ACK, RST); protocolo; portas
Agregadas por dispositivo ou janela	Número de destinos distintos e de fluxos por origem; razão entre tráfego de entrada e de saída

A abordagem tem vantagens claras. Como não inspeciona o *payload*, custa menos e expõe menos dados pessoais, funciona com tráfego cifrado, escala bem e serve para dispositivos que não podem correr *software* de segurança. Em SDN, os contadores de fluxo já existem no controlador, o que facilita o seu uso. Tem também limites. Sem *payload*, a injeção de comandos e o *malware* escondido em tráfego aparentemente normal são difíceis de detetar. A agregação em janelas introduz

atraso: no R-IDPS, por exemplo, a deteção opera sobre janelas de 10 s [20]. E os modelos dependem de *datasets* rotulados, muitas vezes simulados, o que afeta a generalização, como se mostra na QF2 (Secção IV-C).

C. Aprendizagem automática para deteção

Com o aumento de dispositivos IoT em casas e empresas, a exposição a todo o tipo de ataques tem vindo a crescer. Estes ataques podem ser conhecidos, com diferenças já identificadas entre o seu tráfego e o tráfego normal da rede, ou podem ser totalmente novos, sem nenhum rótulo de ataque conhecido.

Devido a esta diferença nos tipos de ataque que podem acontecer, os modelos de ML também diferem, sobretudo na forma como detetam estes ataques e na sua eficácia para cada um deles.

Os modelos que utilizam os dados já conhecidos sobre os vários ataques têm uma aprendizagem supervisionada. Na fase de treino, o modelo já sabe que tráfego é malicioso e procura os padrões que esse tráfego tem em comum, para depois identificar e rotular os ataques em tráfego novo. Um exemplo deste tipo de aprendizagem é a *Random Forest*.

1) Random Forest: A *Random Forest* (RF) [21] é um algoritmo de classificação bastante usado na deteção de ataques, que permite também identificar as *features* de tráfego mais importantes para os detetar. Com isso, é possível reduzir o custo computacional da deteção, já que deixa de ser necessário analisar todas as *features*, mas apenas as indicadas [22].

O algoritmo envolve a construção de uma coleção de árvores de decisão, cada uma treinada com uma amostra aleatória dos dados, que classificam o tráfego como benigno ou malicioso; a classe final é a escolhida pela maioria das árvores. As árvores integram também a ideia de *random node split*: em cada nó, apenas um subconjunto aleatório das *features* de tráfego é considerado.

Com esta ideia aplicada, a RF consegue medir a importância de cada *feature*. Para cada árvore, calcula-se a exatidão (*accuracy*) nos dados que ficaram fora da sua amostra e volta a calcular-se depois de embaralhar, aleatoriamente, os valores de uma das *features*. Se a exatidão cair muito com os novos valores, a *feature* alterada é importante para distinguir o tráfego malicioso e deve ser mantida, para que o modelo detete os ataques com mais sucesso [21], [22].

Apesar de ser muito bom a identificar ataques já conhecidos, este modelo é consideravelmente pior na deteção de ataques novos, para os quais não existem exemplos nem rótulos no treino. Como muitas vezes não existem dados rotulados em tempo real e os ataques mudam constantemente, os modelos que não dependem de rótulos de ataque tornam-se importantes: aprendem o comportamento normal da rede e assinalam o que se afasta dele. Dois são usados neste trabalho.

2) Isolation Forest: O *Isolation Forest* (iForest) [23] é um algoritmo não supervisionado eficiente para identificar anomalias em grandes volumes de tráfego. Ao contrário dos métodos baseados em distâncias ou densidades, aproveita duas características próprias das anomalias: são raras e têm valores muito diferentes do comportamento habitual, o que facilita o seu isolamento.

O algoritmo constrói um conjunto de árvores binárias, as *isolation trees*. Em cada nó escolhe aleatoriamente uma *feature* e um valor de corte entre o mínimo e o máximo dessa *feature*. Como as anomalias precisam de menos partições para ficarem isoladas, aparecem mais perto da raiz, com caminhos mais curtos. Seja $X = \{x_1, \ldots, x_n\}$ um conjunto com n amostras e $h(x)$ o comprimento do caminho de x, isto é, o número de ligações da raiz até à folha onde x termina. O *score* de anomalia é

$$s(x,n) = 2^{-\frac{E[h(x)]}{c(n)}}, \qquad c(n) = 2H(n-1) - \frac{2(n-1)}{n} \quad (1)$$

em que $E[h(x)]$ é a média de $h(x)$ em todas as árvores e $c(n)$ é o comprimento médio de caminho numa árvore binária com n amostras, que serve para normalizar. O número harmónico é aproximado por $H(i) \approx \ln(i) + \gamma$, com $\gamma \approx 0{,}5772$ (a constante de Euler-Mascheroni). O *score* varia em $(0, 1]$: valores próximos de 1 indicam uma amostra isolada com muita facilidade, provavelmente um ataque; valores claramente abaixo de 0,5 indicam tráfego normal; e se todas as amostras tiverem $s \approx 0{,}5$, o conjunto não tem anomalias distintas [23].

3) Autoencoder: O *autoencoder* é uma rede neuronal simétrica treinada para reproduzir a própria entrada. Quando é treinado só com tráfego benigno, aprende a reconstruí-lo bem, e falha na reconstrução de padrões que nunca viu, como os de um DDoS ou de uma *botnet* [24]. Na literatura é muitas vezes classificado como não supervisionado, porque não usa rótulos de ataque; neste trabalho é tratado como semi-supervisionado, porque o treino exige saber que o tráfego usado é benigno.

A rede tem duas partes. O codificador comprime a entrada $x \in \mathbb{R}^d$, com as *features* de um fluxo, numa camada oculta $h \in \mathbb{R}^p$ com $p < d$:

$$h = f(W_1 x + b_1) \quad (2)$$

em que W_1 é a matriz de pesos, b_1 o vetor de *bias* e $f(\cdot)$ uma função de ativação não linear, como a ReLU ou a sigmoide. O descodificador tenta reconstruir a entrada a partir dessa representação:

$$\hat{x} = g(W_2 h + b_2) \quad (3)$$

O desvio entre a entrada e a reconstrução mede-se, para cada amostra, pelo erro quadrático médio. A rede é treinada para minimizar a média deste erro nas N amostras de treino, e uma amostra é classificada como anómala quando o seu erro ultrapassa um limiar θ, definido a partir do erro observado no tráfego benigno de validação:

$$e(x) = \frac{1}{d}\sum_{j=1}^{d}(x_j - \hat{x}_j)^2, \qquad \text{anomalia} \iff e(x) > \theta \quad (4)$$

A diferença entre os dois modelos tem consequências práticas. O *Isolation Forest* tem custo computacional baixo e não precisa de dados de treino rotulados, o que o torna adequado para equipamentos com poucos recursos. O *autoencoder* exige mais capacidade de treino, mas consegue captar relações não

lineares entre as *features*, e é por isso mais usado em análise centralizada de fluxos [24], [25].

D. Mecanismos de resposta e contenção

A deteção de uma intrusão não garante, por si só, que a ameaça seja interrompida. Depois de identificado um comportamento malicioso, é necessário aplicar medidas que impeçam o dispositivo comprometido de continuar a atacar outros equipamentos ou a comprometer o funcionamento da rede. Enquanto um sistema de deteção de intrusões (IDS) pode limitar-se a gerar alertas, um sistema de prevenção (IPS) ou de resposta a intrusões (IRS) procura atuar diretamente sobre a ameaça. Em ambientes IoT, onde nem sempre existe um administrador disponível para intervir, a automatização deste processo torna-se particularmente importante [12], [26].

Uma das formas mais diretas de contenção é o bloqueio e filtragem de tráfego, através de *firewalls* ou listas de controlo de acessos (ACL). Estas permitem restringir comunicações com base em características como o endereço IP, a porta ou o protocolo utilizado. A resposta pode passar pelo descarte de pacotes, pelo bloqueio de determinados fluxos ou pela limitação da taxa de transmissão (*rate limiting*). Uma intervenção mais abrangente consiste no isolamento do dispositivo, impedindo que este continue a comunicar livremente com os restantes equipamentos. Este isolamento pode ser conseguido através de regras de *firewall*, listas negras (*blacklists*) ou segmentação da rede, incluindo a utilização de VLANs de quarentena. A escolha entre uma restrição parcial e o isolamento completo deve considerar a gravidade da ameaça e o impacto sobre as comunicações legítimas [26]–[28].

A contenção pode também ser realizada através de redes definidas por *software* (SDN), que permitem alterar dinamicamente as políticas de encaminhamento e filtragem. Ao separar logicamente o plano de controlo do plano de dados, o SDN possibilita que um controlador central instale regras nos equipamentos de rede, recorrendo, por exemplo, ao OpenFlow. A utilização de *switches* programáveis com P4 acrescenta a possibilidade de definir o processamento dos pacotes diretamente no plano de dados. Estas tecnologias permitem bloquear, redirecionar ou limitar tráfego malicioso sem exigir alterações ao *software* dos dispositivos IoT [3].

O local onde a resposta é aplicada constitui outra decisão importante. A proteção pode funcionar no próprio dispositivo (*host-based*), permitindo-lhe descartar pacotes suspeitos, embora esta abordagem esteja limitada pela capacidade de processamento e memória disponível [12]. Em alternativa, a contenção pode ser executada num *gateway* ou nó *edge*, que concentra as comunicações de vários dispositivos e permite protegê-los sem instalar mecanismos de segurança individuais. Os controladores SDN e *switches* programáveis permitem aplicar políticas em diferentes pontos da infraestrutura, enquanto a *cloud* pode centralizar a análise e a tomada de decisões. Importa, por isso, distinguir o local onde a resposta é decidida daquele onde é efetivamente executada [3], [27].

Por último, a eficácia da contenção depende também de como e quando a resposta é desencadeada. A estratégia mais simples consiste em aplicar uma medida predefinida após a classificação de tráfego como malicioso. Contudo, uma decisão baseada numa única observação pode originar bloqueios indevidos. Algumas abordagens recorrem, por isso, a limiares ou verificações adicionais antes de isolar um dispositivo [28]. Outras utilizam mecanismos de decisão, incluindo aprendizagem por reforço, para escolher ações de acordo com o estado da rede e as consequências esperadas [25], [26]. O momento da intervenção também é relevante, pois agir demasiado tarde pode permitir que o ataque produza danos, enquanto agir com informação insuficiente aumenta o risco de falsos positivos [29]. Assim, um mecanismo de contenção automática deve procurar equilibrar rapidez, fiabilidade e continuidade dos serviços legítimos.

III. Metodologia da revisão sistemática

A revisão segue o protocolo de RSL definido na unidade curricular. O processo teve quatro fases: (1) definição das perguntas de pesquisa; (2) construção da *string* de busca e pesquisa nas bases de dados; (3) seleção dos estudos, com critérios de inclusão e exclusão, leitura pelo método *three-pass* [30] e avaliação de qualidade; e (4) extração dos dados para um formulário comum. Todo o processo, da importação das referências à extração, foi gerido na ferramenta Parsifal. Os ficheiros exportados de cada base, o registo das pesquisas e os dados extraídos estão disponíveis no repositório do projeto [31].

A. Perguntas de pesquisa

Para delimitar o âmbito foi usada a estrutura PIPOC. A população são as redes e os dispositivos IoT. A intervenção é a deteção de dispositivos comprometidos com ML a partir do tráfego de rede. A comparação faz-se entre famílias de modelos e entre mecanismos de resposta. Os resultados de interesse são o desempenho da deteção e a eficácia da contenção. O contexto é a aplicação no *gateway* ou na *edge*, onde o tráfego pode ser observado e bloqueado.

Daqui resultaram uma questão geral (QG), quatro questões focadas (QF1–QF4) e uma questão estatística (QE), apresentadas na Tabela III. A QF1 e a QF2 caracterizam a deteção: que modelos se usam, com que dados são treinados e como são avaliados. A QF3 é o centro da revisão, porque trata da resposta, isto é, do que o sistema faz depois de detetar um dispositivo comprometido. A QF4 avalia se as propostas são exequíveis fora de um *dataset* estático. A QE dá a visão quantitativa do conjunto de estudos.

Tabela III
PERGUNTAS DE PESQUISA

ID	Pergunta
QG	Como é que a análise de tráfego com ML é usada para detetar e conter automaticamente dispositivos IoT comprometidos?
QF1	Que modelos (supervisionados, não supervisionados e semi-supervisionados) e que *features* de tráfego são usados?
QF2	Que *datasets*, tipos de ataque e métricas de avaliação são usados?
QF3	Que mecanismos de resposta automática são propostos, onde são aplicados (dispositivo, *gateway*/*edge*, controlador SDN ou *cloud*) e como são desencadeados?
QF4	As soluções são avaliadas apenas *offline*, com *datasets*, ou também em tempo real ou num protótipo, e com que custo computacional?
QE	Quantas publicações existem por ano, por tipo de modelo e por *dataset*?

As perguntas não ficaram fechadas à partida. Antes da triagem, a *string* principal foi complementada com os termos "systematic literature", "systematic review" e "literature review", para encontrar revisões já publicadas sobre o tema. Esta pesquisa secundária devolveu dez resultados (oito no Scopus e dois no IEEE Xplore), que correspondem a oito revisões distintas. A leitura das três mais próximas do tema [9], [10], [32], complementada por uma revisão anterior sobre a deteção de *botnets* em redes IoT com SDN [8], levou a três ajustes: a QF2 passou a incluir os tipos de ataque; a QF3 passou a perguntar onde a resposta é aplicada e como é desencadeada; e foi acrescentada a QF4, para separar as propostas avaliadas apenas sobre *datasets* das que foram testadas num protótipo ou em tempo real. As revisões encontradas servem de enquadramento e não entram no conjunto de estudos analisados.

B. String de busca e bases de dados

A *string* de busca combina com AND quatro grupos de termos, cada um com os seus sinónimos ligados por OR:

> ("internet of things" OR "IoT") AND ("intrusion detection" OR "anomaly detection" OR "malware detection" OR "botnet detection") AND ("machine learning" OR "deep learning") AND ("mitigation" OR "quarantine" OR "automated response" OR "intrusion prevention" OR "countermeasure")

O primeiro grupo fixa o domínio, o segundo a tarefa de deteção e o terceiro a técnica. O quarto grupo, com os termos ligados à resposta, é o que distingue esta revisão de uma revisão sobre deteção: sem ele, a pesquisa devolve sobretudo estudos que terminam no alerta. Uma primeira versão da *string*, que incluía também termos como "network traffic", "traffic analysis" e "isolation" e era aplicada a todos os campos, devolvia 3184 resultados na ACM e 1910 no IEEE Xplore, na sua maioria fora do tema. A versão final retira os termos mais genéricos e limita a pesquisa ao resumo. A exigência de que a deteção use tráfego de rede passou para os critérios de exclusão (EC3).

A pesquisa foi feita a 6 de outubro de 2026 em cinco bases (Tabela IV). O IEEE Xplore e a ACM Digital Library cobrem as principais conferências e revistas de redes e segurança; o Scopus é um índice multidisciplinar e alarga a cobertura; o ScienceDirect inclui as revistas da Elsevier, como a *Computer Networks*; e a MDPI publica em acesso aberto revistas como a *Sensors*. Em todas foi pesquisado apenas o resumo, para publicações entre 2020 e 2026. No Scopus foram mantidos apenas artigos de revista e de conferência; na ACM foi usada a coleção *ACM Guide to Computing Literature*. O ScienceDirect limita o número de operadores booleanos por pesquisa, pelo que foi usada uma versão reduzida, com os termos mais frequentes de cada grupo:

> "IoT" AND ("intrusion detection" OR "anomaly detection" OR "botnet detection") AND ("machine learning" OR "deep learning") AND ("mitigation" OR "intrusion prevention" OR "automated response")

Tabela IV
BASES DE DADOS PESQUISADAS (6/10/2026)

Base	Filtros	Resultados
IEEE Xplore	resumo, 2020–2026	139
ACM Digital Library	resumo, 2020–2026, ACM Guide	34
Scopus	resumo, 2020–2026, *article* e *conference paper*	342
ScienceDirect	resumo, 2020–2026, *string* reduzida	148
MDPI	resumo, 2020–2026	14
Total		**677**

C. Critérios de inclusão e exclusão

Os critérios usados na seleção estão na Tabela V. Os critérios de inclusão garantem a qualidade mínima e a atualidade das fontes: estudos revistos por pares, em inglês, publicados entre 2020 e 2026. Os critérios de exclusão afastam os estudos que não servem as perguntas de pesquisa. O EC3 exclui as abordagens que dependem de dados recolhidos no próprio dispositivo, como o consumo energético, chamadas ao sistema ou análise de executáveis, porque não podem ser aplicadas num *gateway* que só vê o tráfego. O EC4 afasta as revisões e as propostas puramente conceptuais.

O EC6 não fazia parte do protocolo inicial. Foi acrescentado depois da primeira leitura dos resumos, quando se percebeu que muitos estudos usavam termos como *mitigation* ou *prevention* mas tratavam apenas a deteção, deixando a resposta como trabalho futuro ou referindo-a numa frase. Como a QG pergunta como se deteta *e* se contém, esses estudos não respondem ao objetivo da revisão. O refinamento está previsto no protocolo e ficou registado no repositório.

Tabela V
CRITÉRIOS DE INCLUSÃO (IC) E DE EXCLUSÃO (EC)

ID	Critério
IC1	Publicado em revista, conferência ou *workshop* com revisão por pares
IC2	Escrito em inglês
IC3	Publicado entre 2020 e 2026
IC4	Contém os termos da *string* de busca
EC1	Não responde a nenhuma pergunta de pesquisa
EC2	Não é sobre IoT ou ambientes semelhantes
EC3	A deteção não se baseia em tráfego de rede
EC4	Sem avaliação experimental
EC5	Duplicado
EC6	Não propõe nem avalia um mecanismo de resposta ou contenção automática

D. Processo de seleção

Das 677 referências encontradas, 660 foram importadas para o Parsifal. A diferença corresponde a registos que as bases não deixaram exportar (cinco no ScienceDirect) ou que a ferramenta não conseguiu importar. A remoção de duplicados (EC5) eliminou 169 registos, correspondentes a artigos indexados em mais do que uma base, como o Scopus e o IEEE Xplore, deixando 491 estudos únicos. Estes foram divididos em blocos pelos cinco elementos do grupo, e cada bloco passou pelas etapas seguintes:

1) **Título e resumo.** Aplicação dos critérios IC1–IC4 e EC1–EC4. Os casos duvidosos passaram à etapa seguinte, para não excluir estudos relevantes só pelo resumo.
2) **Resposta automática.** Aplicação do EC6 aos estudos aceites. Quando o resumo não era conclusivo, a decisão foi tomada com o texto completo.
3) **Primeira passagem e avaliação de qualidade.** Leitura de cada artigo pela primeira passagem do método *three-pass* [30]: título, resumo, introdução, títulos das secções e conclusões, complementada pelas secções de método e de resultados necessárias para responder às perguntas de qualidade da Tabela VI.

Cada pergunta de qualidade tem três respostas possíveis, com o valor $q_k \in \{0;\ 0{,}5;\ 1\}$ (Não, Parcialmente, Sim). A pontuação de um estudo e é a soma

$$Q(e) = \sum_{k=1}^{8} q_k(e), \qquad 0 \le Q(e) \le 8, \tag{5}$$

e só foram incluídos os estudos com $Q(e) \ge 5$. As perguntas QA2 a QA7 estão ligadas às perguntas de pesquisa (Tabela VI), o que garante que os estudos incluídos têm informação suficiente para lhes responder. A QA5 é deliberadamente exigente, porque muitos artigos anunciam um mecanismo de resposta mas não o implementam nem apresentam resultados sobre ele. A Fig. 1 resume o número de estudos em cada etapa. Dos 491 estudos únicos, 421 foram excluídos na seleção, pelos critérios EC1–EC4 ou pelo EC6. Dos 70 que chegaram à avaliação de qualidade, 33 ficaram abaixo do limiar. Na revisão final do conjunto, seis dos 37 restantes foram ainda excluídos pelo EC6, por não proporem qualquer mecanismo de resposta, e ficaram 31 estudos incluídos.

Tabela VI
PERGUNTAS DA AVALIAÇÃO DE QUALIDADE

ID	Pergunta	Relação
QA1	Os objetivos e a contribuição do estudo estão claramente definidos?	–
QA2	O modelo de ML e as *features* de tráfego estão descritos com detalhe suficiente para se reproduzir?	QF1
QA3	O estudo usa um *dataset* identificado e público, ou disponibiliza o seu?	QF2
QA4	Reporta métricas além da *accuracy*, como *precision*, *recall*, F1 ou FPR?	QF2
QA5	O mecanismo de resposta ou contenção é descrito e avaliado, e não só mencionado?	QF3
QA6	A solução é testada em tempo real, num protótipo ou num *testbed*, e não só *offline*?	QF4
QA7	Reporta o custo computacional, como latência, CPU ou memória?	QF4
QA8	Discute as limitações do trabalho?	–

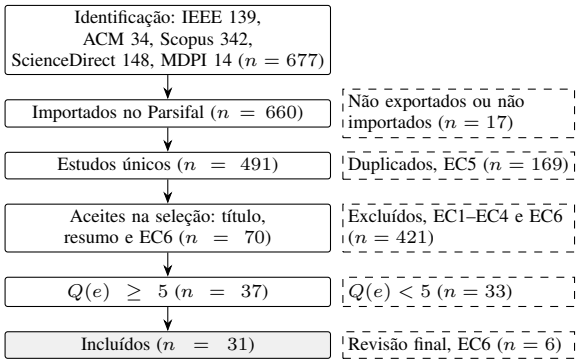


Figura 1. Processo de seleção dos estudos.

E. Extração de dados

Para cada estudo incluído foi preenchido no Parsifal um formulário comum, com campos de texto livre e campos de escolha fixa. Os campos de escolha fixa (tipo de aprendizagem, local da deteção, mecanismo de resposta, tipos de ataque e tipo de avaliação) usam as mesmas categorias para todos os estudos, o que permite contá-los e compará-los diretamente. A Tabela VII mostra como os campos se distribuem pelas perguntas de pesquisa. Os dados extraídos foram exportados do Parsifal para uma folha de cálculo, disponível no repositório [31], a partir do qual foram construídos os gráficos e as tabelas da Secção IV.

Tabela VII
CAMPOS DO FORMULÁRIO DE EXTRAÇÃO

Campos	Pergunta
Referência BibTeX, DOI, ano, autores, tipo e local de publicação	QE
Modelos de ML/DL, tipo de aprendizagem, *features* de tráfego	QF1
Datasets, tipos de ataque detetados, métricas e resultados	QF2
Local da deteção, mecanismo de resposta, forma de desencadear a resposta	QF3
Tipo de avaliação, tempo real, *hardware* de teste, latência e custo computacional	QF4
Principais limitações	Discussão

IV. RESULTADOS DA REVISÃO

A. Visão geral dos estudos (QE)

O conjunto final é composto por 31 estudos, todos com pontuação de qualidade $Q(e) \geq 5$ (Eq. 5). Vinte e quatro foram publicados em revista e sete em conferência. A distribuição por ano (Fig. 2) mostra que o tema é recente e está a crescer: 24 dos 31 estudos são de 2024 ou posterior.

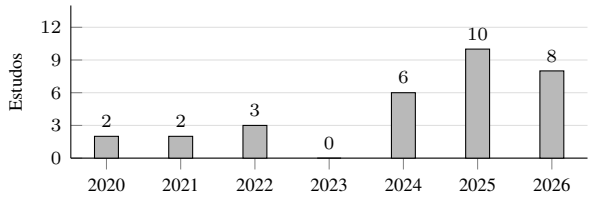


Figura 2. Estudos incluídos por ano de publicação.

Quanto ao tipo de modelo (Fig. 3), 29 dos 31 estudos usam aprendizagem supervisionada, cinco recorrem também a aprendizagem por reforço, em geral para escolher a resposta, três usam aprendizagem semi-supervisionada e só dois incluem modelos não supervisionados. Quanto aos *datasets* (Fig. 4), o UNSW-NB15 e o IoT-23 são os mais usados, em cinco estudos cada, seguidos do ToN-IoT, do Edge-IIoTset e do CICIoT2023, com quatro cada, e 17 estudos recorrem também a dados próprios. As Secções IV-B e IV-C analisam estes resultados em detalhe.

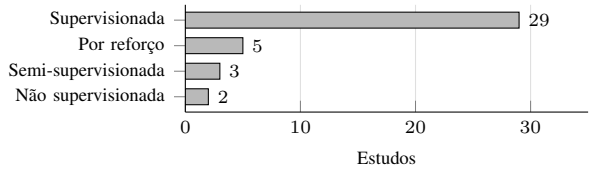


Figura 3. Estudos por tipo de aprendizagem (um estudo pode usar mais do que um).

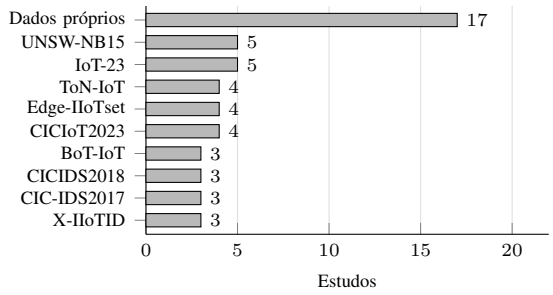


Figura 4. *Datasets* mais usados pelos estudos incluídos.

A Tabela VIII mostra as respostas de cada estudo às oito perguntas de qualidade, e a última linha conta quantos estudos respondem "sim" a cada uma. As pontuações vão de 5,0 a 7,5. Os objetivos estão claros em todos os estudos (QA1) e quase todos reportam métricas além da *accuracy* (QA4). As perguntas mais fracas são, precisamente, as que mais interessam a esta revisão: só 13 dos 31 estudos descrevem *e* avaliam o mecanismo de resposta (QA5), 19 testam a solução em tempo real ou num protótipo (QA6) e 16 reportam o custo computacional (QA7). Na QA6, a resposta "sim" corresponde a testes em *hardware* real, em *testbed* ou em emulação de rede, "parcialmente" a simulações e "não" a avaliações só *offline*. Quando um estudo combina vários tipos de avaliação, conta o mais exigente, e um protótipo conta como "sim", como acontece com o CyberDome, que complementa a simulação com um protótipo [27]. Como os revisores tinham aplicado esta pergunta de forma diferente aos estudos em emulação, a regra foi uniformizada na revisão final. Também a reprodutibilidade (QA2) e a discussão das limitações (QA8) ficam quase sempre pela metade. Durante a leitura foram ainda encontradas incoerências numéricas internas em vários estudos [3], [33]–[38], como valores que mudam entre secções ou métricas que não batem com a matriz de confusão apresentada. Por isso, os valores reportados são usados nesta revisão tal como os autores os publicaram, mas comparados com cautela.

Tabela VIII
AVALIAÇÃO DE QUALIDADE DOS ESTUDOS INCLUÍDOS (S: SIM; P: PARCIALMENTE; N: NÃO)

Estudo	QA1	QA2	QA3	QA4	QA5	QA6	QA7	QA8	$Q(e)$
Karam [15]	S	S	S	S	S	S	S	P	7,5
Toony [4]	S	P	S	S	S	S	S	S	7,5
Al Abdulwahid [39]	S	S	S	S	P	S	P	S	7,0
Iqbal [6]	S	P	S	S	P	S	S	S	7,0
Kponyo [12]	S	S	S	N	S	S	S	S	7,0
Ramirez-Martinez [40]	S	P	S	S	S	S	S	P	7,0
Shukla [41]	S	S	S	S	P	P	S	S	7,0
Srilekha [42]	S	P	S	S	S	S	S	P	7,0
Andrade [34]	S	P	S	S	S	S	S	N	6,5
Gowtham [5]	S	P	P	S	S	S	S	P	6,5
Hussain [36]	S	S	S	S	P	N	S	S	6,5
Mazhar [20]	S	P	P	S	S	S	S	P	6,5
Rasheed [43]	S	P	S	S	P	P	S	S	6,5
Ravi [2]	S	P	P	S	S	S	S	P	6,5
Rose [26]	S	P	S	S	S	S	P	P	6,5
Arulkumar [44]	S	S	S	S	P	N	S	P	6,0
El-Sayed [3]	S	P	S	S	P	S	P	P	6,0
Garba [45]	S	P	S	S	P	S	P	P	6,0
Ibrahim [33]	S	P	S	S	P	P	S	P	6,0
Saurabh [46]	S	S	S	S	P	N	P	S	6,0
Tariq [47]	S	P	S	S	P	S	P	P	6,0
Tawfik [48]	S	P	S	S	P	S	N	S	6,0
Verma [49]	S	S	S	S	P	P	P	P	6,0
Chaudhary [50]	S	S	S	S	P	N	P	P	5,5
Dange [35]	S	P	S	S	N	S	P	P	5,5
Hunde [28]	S	P	P	S	S	P	P	P	5,5
Premakumari [37]	S	P	P	P	S	P	S	P	5,5
Zolotukhin [25]	S	P	S	P	S	S	N	P	5,5
Karibandi [27]	S	P	N	S	P	S	P	P	5,0
Kumar e Kumar [38]	S	P	S	S	P	N	P	P	5,0
Rawat [51]	S	S	S	S	N	N	N	S	5,0
N.º de "S" (31)	31	10	25	28	13	19	16	10	

B. Modelos e features (QF1)

A abordagem supervisionada é a mais comum nos estudos analisados. Os modelos clássicos continuam presentes, sobretudo a RF, as árvores de decisão, o KNN, a regressão logística (LR), o Naive Bayes e a *Support Vector Machine* (SVM) [12], [15], [20], [34]–[36], [40], [45], [46], [49], [51], ao lado de redes profundas como CNN, LSTM, GRU e CNN-BiLSTM [26], [33], [42], [48].

Muitos estudos utilizam vários modelos, cada um de forma diferente: o *stacking* de seis classificadores com um MLP como meta-modelo [3] ou um *ensemble* ponderado de seis classificadores [4]; a combinação de SVM, RF, J48, *K-Nearest Neighbor* (KNN) e *Extreme Gradient Boosting* (XGBoost) num único sistema de deteção de ataques [50]; a junção de XGBoost com LR num meta-classificador, num sistema de deteção multinível que integra um *stacked ensemble* [44]; uma rede neuronal de grafos (GNN) combinada com LSTM para analisar as relações espaciais e temporais de um sistema ciber-físico (CPS) [43], ou um *graph transformer* com o mesmo fim [41]; um classificador baseado em RF para detetar os ataques, seguido de um classificador MLP especializado que identifica o tipo de ataque [6]; Extra Trees no *edge* e CNN-LSTM na *cloud* [42]; uma Bi-GRU com atenção [38]; uma rede Ghost-SE treinada com aprendizagem federada [5]; XGBoost [27]; e uma rede neuronal profunda combinada com limiares dinâmicos [28]. Cinco estudos usam ainda aprendizagem por reforço para escolher a resposta [5], [25], [37], [38], [44].

As *features* de tráfego são um conceito importante para a deteção de anomalias e de ataques em todos estes estudos e, por esta razão, os artigos apresentam as *features* que o modelo considerou importantes durante a sua fase de teste. Al Abdulwahid, por exemplo, destaca o tamanho dos pacotes (*payload size*), o tempo entre chegadas de pacotes (*inter-arrival time*), a duração do fluxo (*flow duration*), o número de tentativas de ligação falhadas, a entropia dos IP de origem e de destino e o protocolo utilizado [39]; Arulkumar e K, a frequência de comandos PLC, os picos de transmissão de dados e as anomalias de autenticação [44]. De forma geral, a maioria dos estudos trabalha com estatísticas de fluxo, como contagens de pacotes e de *bytes*, duração e intervalos entre pacotes, calculadas por fluxo ou por janelas de tempo curtas, de 0,5 s a 2 s [2], [3], e três estudos calculam as *features* no próprio plano de dados, em *switches* P4 [3], [34], [40].

As abordagens que não dependem só de rótulos são minoritárias. Ravi e Shalinie [2] combinam uma rede neuronal com K-means num esquema semi-supervisionado que treina com apenas 10% dos dados rotulados e marca como incertos os casos em que os modelos discordam. Zolotukhin *et al.* [25] usam *autoencoders*, mapas auto-organizados e BiGAN para a deteção e um agente de aprendizagem por reforço para decidir a resposta. Tariq e Ahanger treinam um *autoencoder* empilhado sem rótulos antes de um classificador GRU [47], Shukla e Agarwal usam pré-treino auto-supervisionado [41], e Al Abdulwahid opta por uma aprendizagem semi-supervisionada, que combina *autoencoders* para a deteção de anomalias com redes neuronais recorrentes (RNN) para a análise temporal, detetando anomalias tanto imediatas como de longo prazo [39]. Mesmo nos estudos que juntam modelos supervisionados e não supervisionados [25], [39], [47], os dois são usados em conjunto ou a escolha entre eles fica a cargo de um agente de reforço, sem uma análise do efeito de cada um na contenção. Essa comparação é um dos objetivos da solução proposta na Secção V.

As aplicações destes modelos distinguem-se pelos recursos de *hardware* e pela complexidade do tráfego analisado. Os modelos leves, como o KNN ou as árvores de decisão, são os que cabem nos próprios dispositivos ou em *gateways* de baixo custo. É o caso do mecanismo de Kponyo *et al.* [12], que deteta *floods* SYN, ICMP e UDP diretamente em dispositivos IoT com poucos recursos. O modelo, um KNN treinado fora do dispositivo, classifica o efeito de um afluxo de pacotes na CPU e na memória e ocupa em média 23 MB. Os modelos profundos, como os *autoencoders* ou as redes CNN e LSTM, exigem mais capacidade de treino e aparecem sobretudo em arquiteturas centralizadas, em controladores SDN ou em servidores na *cloud*, onde analisam os dados agregados de vários dispositivos [25], [26], [48]. Nenhum dos 31 estudos usa o *Isolation Forest*, apesar de ser leve e de não precisar de rótulos. Essa ausência, junto com a complementaridade entre modelos leves para a reação imediata e modelos profundos para a análise centralizada, é uma das razões para a solução proposta comparar o *Isolation Forest* e o *autoencoder* no mesmo *gateway*.

Em síntese, a aprendizagem supervisionada é a mais comum entre os estudos analisados; alguns poucos seguem a semi-supervisionada, menos ainda a não supervisionada, e alguns aproveitam ideias da aprendizagem por reforço nos seus modelos [5], [25], [37], [38], [44]. Da mesma forma, há uma grande diferença entre as *features* de tráfego consideradas importantes pelos vários modelos, e diversos estudos não as indicam, ou explicam apenas como o modelo decidiu quais eram as mais importantes, sem nunca as listar [43], [50]. Das que foram indicadas, algumas das mais comuns são o tamanho dos pacotes, que se altera quando o dispositivo envia código malicioso ou comandos fora do padrão; o tempo entre chegadas dos pacotes, que num ataque DDoS se torna muito curto, por causa do volume de pacotes enviados; e a duração do fluxo, que pode indicar comunicação maliciosa se for muito diferente da habitual entre os dispositivos.

C. Datasets, tipos de ataque e métricas (QF2)

Esta secção responde à QF2 com base nos 31 estudos incluídos. Os valores apresentados são os reportados por cada artigo.

***Datasets* utilizados.** A Tabela IX detalha os *datasets* de cada estudo. Muitos combinam *datasets* públicos com dados próprios, gerados em *testbed*, emulação ou simulação: 17 dos 31 estudos usam dados próprios, 13 usam apenas *datasets* públicos e um não identifica o *dataset* [27]. O UNSW-NB15 e o IoT-23 [11] são os mais usados (cinco estudos cada), seguidos do ToN-IoT, do Edge-IIoTset e do CICIoT2023 (quatro cada). A dispersão é grande: os estudos usam 29 *datasets* públicos diferentes, 15 dos quais aparecem num único estudo.

Treze dos 31 estudos usam pelo menos um *dataset* genérico de intrusão, que não é de IoT, como o UNSW-NB15, o CICIDS2017 e 2018, o NSL-KDD, o KDDCUP'99, o CICDDoS2019 ou o CTU-13, o que limita a sua representatividade para IoT. Os dados próprios raramente são públicos: só o MQTTEEB-D [15] e o *dataset* do IDERES [26] estão num repositório aberto, e o do SAE-GRU apenas em parte [47]. Os restantes estão disponíveis a pedido ou não estão disponíveis, o que dificulta a replicação. Vários estudos também não descrevem o tamanho dos dados nem a divisão entre treino e teste [4], [5], [38], [47].

Tipos de ataque detetados. O DoS/DDoS está presente em 28 dos 31 estudos, e em sete é o único alvo [5], [12], [20], [34], [40], [45], [50]. Seguem-se as *botnets* e a comunicação de C&C (15 estudos), o *malware* (13) e a força bruta (9). A Tabela X mostra a distribuição por estudo.

Os ataques detetados não coincidem com aqueles contra os quais a resposta automática é avaliada. Vários estudos listam muitos tipos de ataque porque o *dataset* os contém, mas demonstram a resposta sobretudo contra *floods* SYN, UDP, ICMP ou HTTP. A deteção de ataques *zero-day* só é testada explicitamente em dois estudos: Shukla e Agarwal obtêm uma AUROC de 0,963 [41], e no IDERES só três de 16 ataques *zero-day* são mitigados [26]. Gowtham *et al.* limitam-se a invocá-la [5].

Métricas de avaliação e resultados. As métricas usadas derivam da matriz de confusão e estão definidas nas Eqs. (9)–(11). A *accuracy* é reportada em 28 dos 31 estudos e o F1 em 25, mas só 13 dão um valor explícito da taxa de falsos positivos [3]–[6], [26], [38], [39], [41]–[44], [49], [50], e poucos medem a eficácia da resposta. Os valores da Tabela XI são os que cada estudo reporta e não são comparáveis entre si, porque os *datasets*, as divisões e as tarefas (binária ou multiclasse) diferem.

As métricas de classificação concentram-se acima de 97%, mas a robustez é pouco demonstrada. Quando Karam *et al.* testam o modelo noutro *dataset*, a *accuracy* cai para 8–21% [15]; Hussain *et al.* obtêm um F1 de 0,751 no NSL-KDD, contra cerca de 1,0 nos restantes *datasets* [36]; e no trabalho de Dange e Nitnaware a *accuracy* passa de 99,97% no BoT-IoT para 92% no *testbed* [35].

A eficácia da resposta, central para a questão geral, só tem métrica própria em dez dos 31 estudos: a AME de Toony *et al.* [4], a eficiência de resposta de Srilekha *et al.* [42], a taxa de mitigação de Gowtham *et al.*, de 91,9 a 94,5% [5], e de Premakumari *et al.* [37], a disponibilidade sob ataque de Shukla e Agarwal [41], a taxa de sucesso dos ataques de Ibrahim *et al.* [33], a recompensa de Zolotukhin *et al.*, que soma o tráfego legítimo que passa e o malicioso que é bloqueado [25], os cenários mitigados do IDERES [26] e o efeito na entrega de pacotes e na latência do RankShield [28] e a mitigação de *ransomware* antes da cifra de Al Abdulwahid [39]. Kumar e Kumar, por exemplo, propõem uma mitigação mas não medem a sua eficácia [38]. Foram ainda identificadas incoerências internas em alguns estudos, entre o resumo e as tabelas [36]–[38] ou em contagens que mudam ao longo do texto [20], o que constitui uma limitação das fontes.

Síntese e lacunas. Os resultados reportados são muito altos, mas assentam em *datasets* e métricas pouco comparáveis e raramente medem a contenção. Destacam-se quatro lacunas. A primeira é que os *datasets* são pouco representativos: cerca de 40% dos estudos usam *datasets* genéricos de intrusão, e os dados próprios são quase sempre simulados ou emulados e não são públicos. A segunda é que a generalização raramente é testada fora do *dataset* de treino, e quando o é os resultados caem. A terceira é que os ataques se concentram em *floods*: o DoS/DDoS domina, a resposta avaliada é quase sempre contra *floods*, e as *botnets* com C&C e a força bruta têm pouca cobertura. A quarta é que as métricas de resposta são escassas: cerca de um terço dos estudos mede a eficácia da contenção ou o seu efeito no tráfego legítimo, e menos de metade dá um valor explícito da taxa de falsos positivos. Por isso a solução proposta reporta a FPR e mede as quarentenas indevidas com tráfego só benigno (cenário S1, Secção V).

D. Mecanismos de resposta automática (QF3)

A análise dos estudos selecionados permitiu identificar diferentes abordagens para a resposta automática a ataques em redes IoT. Os mecanismos encontrados incluem o descarte de pacotes, o bloqueio de fluxos, o isolamento de dispositivos e a reconfiguração dinâmica de políticas de segurança. As princi-

Tabela IX
DATASETS USADOS POR ESTUDO

Estudo	*Datasets* públicos	Dados próprios ou simulados
Al Abdulwahid [39]	CICIDS2017, CSE-CIC-IDS2018, UNSW-NB15	Nenhum
Andrade [34]	CICDDoS2019	Nenhum
Arulkumar [44]	DAPT2020, IoT Intrusion Detection, KDDCUP'99, CICIoT2023	Nenhum
Chaudhary [50]	BoT-IoT	Nenhum
Dange [35]	BoT-IoT, CTU-13	*Testbed* próprio, a pedido
El-Sayed [3]	ToN-IoT, Edge-IIoTset, CICIoT2023	Nenhum
Garba [45]	UNSW-NB15, CICDDoS2019	Captura de 24 h em *testbed*, a pedido
Gowtham (FedMP-LDL) [5]	Edge-IIoTset, CIC-IDS2017 (só validação cruzada)	*Testbed* Mininet SDN-*fog*, não disponibilizado
Hunde (RankShield) [28]	–	Simulação em Cooja com ataques ao RPL, 70/30
Hussain [36]	NSL-KDD, MedBIoT, CIC-IDS2017, Edge-IIoTset, IoT-23, ToN-IoT	Nenhum
Ibrahim [33]	IoTForge Pro, WUSTL-IIoT-2021, X-IIoTID	Nenhum (reprodução dos *datasets*)
Iqbal (DeepGuard) [6]	WUSTL-IIoT-2021, X-IIoTID	Nenhum
Karam (ISAAF) [15]	MQTTset	MQTTEEB-D: *testbed* real de IoT de saúde, 222 813 registos, público no Mendeley
Karibandi (CyberDome) [27]	–	*Dataset* com 11 classes, de origem não identificada
Kponyo [12]	–	Medições de CPU e memória no *testbed* (30 384 amostras), não públicas
Kumar e Kumar [38]	CICIoT2023	Nenhum
Mazhar (R-IDPS) [20]	–	*Testbed* Mininet-WiFi com MQTT e ONOS, não público
Premakumari [37]	AWID, IoT-23, WSN-BFSF (uso pouco claro)	Simulação em MATLAB Simulink com 50 nós
Ramirez-Martinez [40]	CICIoT2023; IoT-23 (só tráfego benigno)	Tráfego Slowloris próprio, a pedido
Rasheed (XAI-CPS) [43]	SWaT, WADI, BATADAL, IIoT-Security	Nenhum
Ravi [2]	NSL-KDD	*Testbed* com 115 200 registos (10% rotulados), não público
Rawat [51]	IoT-23	Nenhum
Rose [26]	–	913 PCAPs e imagens, públicos no IEEE DataPort
Saurabh [46]	UNSW-NB15	Nenhum
Shukla [41]	UNSW-NB15, CICIDS2018, BoT-IoT	Smart-ITS, simulado com SUMO, origem pouco clara
Srilekha [42]	UNSW-NB15 (aumentado com padrões IoT)	Telemetria IoT gerada com anomalias sintéticas
Tariq (SAE-GRU) [47]	IoT-23, MedBIoT	Emulação de cidade inteligente (mais de 200 nós), parcialmente no Zenodo
Tawfik [48]	IoTID20	Tráfego emulado em Mininet, não público
Toony (MULTI-BLOCK) [4]	Edge-IIoTset, ToN-IoT, X-IIoTID	Tráfego emulado em Mininet-WiFi para os testes de ataque, a pedido
Verma (PETRAK) [49]	KDDCUP'99, CIC-MalMem-2022, ToN-IoT, CICEV2023, VeReMi	Simulação em NS-3 com 697 792 registos, a pedido
Zolotukhin [25]	CICIDS2018	Nenhum

Tabela X
TIPOS DE ATAQUE DETETADOS POR ESTUDO (DOS: DOS/DDOS; BOT: BOTNET E C&C; FB: FORÇA BRUTA; MAL: MALWARE)

Estudo	DoS	Bot	FB	Mal	Outros ataques
Al Abdulwahid [39]	x	x	x	x	
Andrade [34]	x				Não
Arulkumar [44]	x	x			Ameaças persistentes avançadas (APT)
Chaudhary [50]	x				Não
Dange [35]		x			Varrimento
El-Sayed [3]	x		x	x	Varrimento, *spoofing*, MITM, injeção *web*
Garba [45]	x				Não (SYN *flood* com IP falsificado)
Gowtham [5]	x				Não
Hunde [28]					Ataques ao *rank* do RPL
Hussain [36]	x	x		x	MITM, *spoofing*, reconhecimento
Ibrahim [33]	x		x	x	Varrimento, reconhecimento, XSS
Iqbal [6]	x	x		x	
Karam [15]	x		x		MQTT: Slowite, dados malformados, *publish flood*
Karibandi [27]	x	x	x	x	Varrimento, outros
Kponyo [12]	x				Não (*floods* ICMP, UDP, SYN)
Kumar e Kumar [38]	x			x	SQLi, *buffer overflow*
Mazhar [20]	x				Não (*floods* ICMP e SYN)
Premakumari [37]	x				*Black-hole*, *wormhole*, *selective forwarding*, injeção de dados
Ramirez-Martinez [40]	x				Não (Slowloris)
Rasheed [43]	x	x			
Ravi [2]	x				Varrimento; R2L e U2R (NSL-KDD)
Rawat [51]		x			Não
Rose [26]	x	x	x	x	*Exploits* de serviços, enumeração
Saurabh [46]	x			x	Reconhecimento, *fuzzers*
Shukla [41]	x	x	x	x	Reconhecimento, exfiltração, *zero-day*
Srilekha [42]	x			x	Varrimento, *insider*, *fuzzers*
Tariq [47]	x	x	x	x	Varrimento, *spoofing*, exfiltração, escuta
Tawfik [48]	x	x			Varrimento, MITM (ARP *spoofing*)
Toony [4]	x	x			*Spoofing*, MITM, varrimento, SQLi, XSS
Verma [49]	x	x		x	*Backdoor*, injeção, *ransomware*, varrimento
Zolotukhin [25]	x	x	x		XSS, SQLi
Total (31 estudos)	28	15	9	13	

Tabela XI
MÉTRICAS REPORTADAS E MELHORES RESULTADOS POR ESTUDO (P: PRECISION; R: RECALL; ACC: ACCURACY)

Estudo	Métricas reportadas	Melhor resultado reportado
Al Abdulwahid [39]	Acc, P, R, F1, FPR	Acc 96,1%, F1 95,0%, FPR 3,2%
Andrade [34]	Acc, R, F1	NB: Acc 98,04%, F1 98,98% (*offline*)
Arulkumar [44]	Acc, P, F1, especificidade, sensibilidade, NPV, MCC, FPR, FNR	Acc 97,67% (divisão 70:30) e 98,92% (outra divisão); FPR 0,0227
Chaudhary [50]	Acc, P, R, F1, FPR	RF: Acc 98,89%, F1 97,74%, FPR 0,006
Dange [35]	Acc, P, R, F1	SVM: Acc 99,97% no BoT-IoT; 0,92 no *testbed*
El-Sayed [3]	Acc, F1, FPR, AUC	CICIoT2023: Acc 99,32%, F1 99,23%, FPR 0,49%
Garba [45]	Acc, P, R, F1	DT: 99,57% (próprio), 98,20% (UNSW-NB15), 99,95% (CICDDoS2019)
Gowtham [5]	Acc, P, R, F1, AUC, FPR	Acc 99,72% (*dataset* não explicitado); 96,87% no *testbed*; FPR 0,0032% (0,83% no *testbed*)
Hunde [28]	Acc, P, R, F1; latência, energia e PDR	Acc até 99,8%, F1 até 99,7%; PDR +67%, latência −66%
Hussain [36]	F1, Acc, P, R	RF: F1 0,9998–1,000 em cinco *datasets*, mas 0,751 no NSL-KDD (média 0,957)
Ibrahim [33]	Acc, F1, taxa de sucesso dos ataques	Acc 97,79–99,76%; F1 99,1% (multiclasse); sucesso dos ataques 0–8,79%
Iqbal [6]	Acc, FPR	Acc 99,99% (*desktop*), 99,82% (Raspberry Pi 5); FPR 0,2%
Karam [15]	Acc, P, R, F1, AUC	Acc 0,851 no MQTTset; cai para 0,08–0,21 noutro *dataset*; 0,87 após novo treino
Karibandi [27]	Acc, F1 (macro)	Acc 99,99%, F1 0,999
Kponyo [12]	Acc	KNN: 100%
Kumar e Kumar [38]	Acc, P, sensibilidade, especificidade, F, FPR, FNR	Acc 0,996, FPR 0,01753; sem métrica de eficácia da mitigação
Mazhar [20]	P, R, F1, taxa de deteção por n.º de *bots*	P 100%, R 0,97–0,98, F1 0,98–0,99; FP = 0
Premakumari [37]	Energia, PDR, latência, Acc de deteção, eficiência de mitigação	Deteção 93–98%, mitigação 85–95%; sem P, R, F1 nem FPR
Ramirez-Martinez [40]	Acc, P, R, F1	RF: Acc 97,02% *offline* e 98,28% no *testbed*; F1 97,21% (Slowloris)
Rasheed [43]	P, R, F1, FPR	F1 0,900–0,956 e FPR 2,12–4,67% nos quatro *datasets*
Ravi [2]	Acc, taxa de deteção, F, MCC	NSL-KDD: Acc 99,78%, F 99,72%, MCC 99,44%
Rawat [51]	Acc, P, R, F1	KNN: Acc 92,68%, F1 0,85
Rose [26]	Acc, P, R, F1, FPR	CNN: Acc 98,35%, F1 99,16%, FPR 0,98%; 13 cenários mitigados
Saurabh [46]	Acc, P, R, F1	RF: Acc 98,4%, F1 0,98
Shukla [41]	Acc, P, R, F1, FAR, macro-F1, AUROC, disponibilidade	Acc 97,46–98,64%, F1 96,59–98,04%, FAR 0,88–1,41%; *zero-day*: AUROC 0,963
Srilekha [42]	Acc, P, FPR, eficiência de resposta	Acc 99,2%, FPR cerca de 1,5% (lidos de gráficos)
Tariq [47]	Acc, P, R, F1, AUC, validação cruzada (10 partes)	Acc 98,65%, F1 98,27%, AUC 0,993; FPR só em gráfico
Tawfik [48]	Acc, F1	LSTM: Acc 98,3%, F1 85,9% (binário)
Toony [4]	Acc, P, R, F1, AUC, MCC, FPR, AME	Acc 99,75% (Edge-IIoTset), 98,17% (ToN-IoT), 98,21% (X-IIoTID); AME 88,29–99,95%; FPR 0,40–5,88%
Verma [49]	Acc, TPR, FPR, sensibilidade, especificidade	Acc 99,70–99,998% (três *datasets*), 98,8% no ToN-IoT; FPR 0,000–0,003
Zolotukhin [25]	TPR, FPR e recompensa (só em curvas)	Sem valores numéricos

Tabela XII
TIPO DE AVALIAÇÃO, AMBIENTE E CUSTO COMPUTACIONAL (QF4)

Estudo	Avaliação	Ambiente / *hardware*	Custo e tempos reportados
Kponyo [12]	Tempo real	4 Raspberry Pi 3B (dispositivos)	Deteção 0,10 s; mitigação 0,22 s; 21–28 MB; +35,8 mW
Ravi [2]	*Testbed* real	Cisco Nexus (*fog*), 30 Raspberry Pi	Treino 1986 s; teste 6,6 s; resposta ≥34% mais rápida que na *cloud*
Dange [35]	*Testbed* real (ML *offline*)	Raspberry Pi 4, ThingsBoard	Só tempo de treino
Karam [15]	*Hardware* real e *offline*	Raspberry Pi 4 (*edge*), *broker* MQTT, ThingsBoard	Deteção ≈17 s; mitigação 1–3 s; CPU do *broker* 60–80% sob DoS
Srilekha [42]	*Hardware* real e simulação	Raspberry Pi 4, AWS EC2, Hyperledger Fabric	Resposta <3 s; *blockchain* no Raspberry Pi: +15% de CPU, +50 MB
Garba [45]	*Hardware* real	Raspberry Pi (OpenFlow), Ryu	Só tempos de treino e teste (2 min a 1 h 40)
Iqbal [6]	*Hardware* real e *offline*	Raspberry Pi 5 (10 W); *desktop* i7	Latência <30 ms
Al Abdulwahid [39]	Tempo real (servidor)	i9, 64 GB, RTX 3090	Latência 12,8 ms; 275 MB
Andrade [34]	*Testbed* 5G	Open5GS, UERANSIM, P4, ONOS	Deteção 20–140 s; mitigação 2–4 s; CPU 2–60%
Rose [26]	*Testbed* virtual	VMs (Cyber-Trust)	Só tempo de treino
Ramirez-Martinez [40]	Emulação	Mininet, BMv2, VM 4 *cores*	Deteção 20,6 s; mitigação 0,128 s
El-Sayed [3]	Emulação	Mininet-WiFi, P4	Deteção 11–33 ms; CPU 20–35%
Zolotukhin [25]	Emulação	VMs, OVS, OpenDaylight	Não reportado
Tawfik [48]	Emulação e *offline*	Mininet e Ryu num Raspberry Pi 3B; IoTID20	Não reportado
Toony [4]	Emulação	Mininet-WiFi, BMv2, Ryu; VM i7	Deteção 2–19 ms; mitigação ≈3 ms; CPU 100% sob ataque
Gowtham [5]	Emulação	Mininet SDN-*fog*; i5 e 20 Raspberry Pi 4	Inferência 196 ms sob ataque; CPU do *fog* 23%; controlo 42–51 ms
Mazhar [20]	Emulação	Mininet-WiFi, ONOS	Agente no dispositivo: +0,4 pp de CPU, +0,3 pp de memória
Tariq [47]	Emulação e *offline*	Servidores Xeon; nós *edge* com GPU	Não reportado em valores
Ibrahim [33]	Simulação	Portátil i5, 4 GB	Inferência 1,6 ms; CPU 14,5%; 448 MB
Shukla [41]	Simulação	SUMO; RTX 3090	Resposta 34,9 ms (deteção 18,6; mitigação 16,3); 6,4 GB de GPU
Verma [49]	Simulação e *offline*	NS-3; Weka num i7	Só tempos de construção e teste do modelo
Premakumari [37]	Simulação	MATLAB Simulink, 50 nós	Latência 120 ms; energia 45–74 J
Karibandi [27]	Simulação e protótipo	*Cloud*, *gateways* e SDN	Inferência 8,74 µs por amostra; treino 39,6 s
Hunde [28]	Simulação	Cooja (Zolertia Z1), 19–100 nós	Treino 218–2750 ms
Rasheed [43]	*Offline* e simulação	GPU A100	Latência 15,6 ms; 2,4 GB de RAM; mitigação média em 47,8 s (simulação)
Saurabh [46]	*Offline*	UNSW-NB15	Tempo por modelo (DT 4 ms; RF 46 ms)
Rawat [51]	*Offline*	IoT-23	Não reportado
Hussain [36]	*Offline*	Google Colab	*Handshake* pós-quântico 3,1–4,4 s
Kumar e Kumar [38]	*Offline*	PC i3, 8 GB	Treino 250 s; teste 15 s
Arulkumar [44]	*Offline*	i7, 16 GB, RTX	Latência 200 ms
Chaudhary [50]	*Offline*	i5 e Azure	115–170 s por modelo

pais diferenças entre os estudos estão no grau de intervenção, no componente responsável pela execução da resposta e nas condições que levam à sua ativação.

O bloqueio de tráfego é uma das medidas mais diretas. Kponyo *et al.* [12] apresentam uma solução que funciona no próprio dispositivo IoT, utilizando um classificador k-NN para identificar ataques DoS e o NetfilterQueue para descartar os pacotes associados à ameaça. Neste caso, a deteção e a resposta são realizadas localmente, sem depender da intervenção de um controlador central. Em contraste, o MP-GUARD, de El-Sayed *et al.* [3], combina aprendizagem automática com uma arquitetura SDN de múltiplos controladores e *switches* P4. A identificação de tráfego suspeito permite desencadear políticas de filtragem, limitação de débito e isolamento de recursos na infraestrutura de rede. Estas abordagens mostram que o bloqueio pode ser aplicado tanto no equipamento protegido como em pontos intermédios por onde passam as suas comunicações.

Uma estratégia mais abrangente é apresentada pelo CyberDome, de Karibandi *et al.* [27]. O sistema combina um IDS baseado em XGBoost, executado na *cloud*, com um módulo de mitigação baseado em SDN. A resposta é organizada em três níveis: controlo dinâmico de fluxos, segmentação da rede e quarentena do dispositivo. Quando é detetada atividade maliciosa, o sistema pode começar por restringir o tráfego e aumentar progressivamente o nível de isolamento caso a ameaça persista. Esta abordagem distingue-se por permitir aplicar medidas com diferentes níveis de severidade, em vez de recorrer imediatamente ao bloqueio total. Embora os autores demonstrem os diferentes níveis de mitigação numa rede simulada, a avaliação quantitativa concentra-se sobretudo no desempenho do classificador, não permitindo determinar com o mesmo detalhe a eficácia e a latência das medidas de contenção.

O RankShield, proposto por Hunde e Ismail [28], apresenta outro mecanismo de isolamento, direcionado para ataques ao protocolo de encaminhamento RPL. O sistema utiliza uma rede neuronal profunda (DNN) para identificar alterações anormais dos valores de rank anunciados pelos dispositivos. Antes de executar a resposta, essa previsão é sujeita a uma verificação adicional baseada em limiares dinâmicos, procurando distinguir ataques de alterações legítimas na topologia. Quando a atividade maliciosa é confirmada, o nó atacante é colocado numa lista negra, cuja informação é propagada pela rede para impedir a sua participação no encaminhamento. Ao contrário de soluções que dependem de *firewalls* ou controladores SDN, o RankShield aplica a contenção através dos próprios mecanismos de comunicação da rede RPL. A validação adicional constitui também uma forma de reduzir o risco de isolamento indevido.

Estas diferenças evidenciam que o local de aplicação da resposta depende da arquitetura e do tipo de ameaça. A solução de Kponyo *et al.* [12] atua no próprio dispositivo, enquanto o MP-GUARD [3] utiliza controladores e *switches* programáveis. O CyberDome [27] centraliza a deteção e a gestão na *cloud*, mas recorre à infraestrutura SDN para aplicar as restrições. Já o RankShield [28] adota uma abordagem distribuída, em que os nós da rede participam no isolamento dos atacantes. As soluções centralizadas facilitam a gestão de políticas para vários equipamentos, ao passo que as abordagens locais ou distribuídas podem reduzir a dependência de um único componente. Contudo, estas últimas exigem que os dispositivos envolvidos tenham capacidade para executar os mecanismos de proteção.

Relativamente ao desencadeamento da resposta, os estudos apresentam estratégias que vão desde a aplicação de regras predefinidas até à seleção dinâmica de contramedidas. No IDERES, Rose *et al.* [26] combinam deteção de intrusões com grafos de ataques e um modelo de decisão probabilístico baseado em POMDP (Partially Observable Markov Decision Process). Este mecanismo considera os alertas recebidos, a informação disponível sobre a rede e os possíveis caminhos de ataque para selecionar regras de *firewall*. A resposta pode consistir no bloqueio de uma comunicação específica ou numa restrição mais abrangente das ligações de um equipamento. Desta forma, procura-se evitar que todas as ameaças desencadeiem automaticamente o mesmo tipo de bloqueio, independentemente do seu contexto.

Uma abordagem adaptativa é também apresentada no Curious SDN, de Zolotukhin *et al.* [25], que utiliza aprendizagem por reforço para modificar dinamicamente as políticas de segurança. O agente observa o estado da rede e pode tomar decisões como alterar regras de fluxo, redirecionar tráfego ou ajustar a sensibilidade dos mecanismos de deteção. A aprendizagem procura equilibrar a quantidade de tráfego malicioso bloqueado com a preservação das comunicações legítimas. No entanto, os resultados experimentais mostram que o desempenho da política aprendida depende do cenário, por exemplo, a abordagem apresenta resultados favoráveis em ataques DDoS e *botnets*, mas revela dificuldades em determinados ataques a aplicações *web*. Isto demonstra que a utilização de mecanismos inteligentes não garante, por si só, respostas eficazes em todas as situações.

A validação experimental dos mecanismos de contenção constitui uma limitação importante na comparação dos estudos. O MP-GUARD [3] e o Curious SDN [25] recorrem a ambientes de rede emulados, enquanto o RankShield [28] é avaliado através de simulações com o simulador Cooja. O IDERES [26] utiliza um testbed virtualizado e apresenta resultados de mitigação para diferentes cenários, mas a eficácia varia conforme o ataque. O trabalho de Kponyo *et al.* [12] distingue-se pela utilização de dispositivos Raspberry Pi, nos quais são efetivamente executadas ações de descarte de pacotes. Já o CyberDome [27] apresenta uma contribuição importante para a arquitetura da resposta, embora a avaliação da execução operacional da contenção seja menos aprofundada. A Secção IV-E aprofunda estas diferenças quanto aos ambientes de teste, tempos de resposta e custos computacionais.

Em conjunto, os estudos mostram que a deteção automática não deve ser confundida com a contenção efetiva. Um modelo pode apresentar uma elevada exatidão sem que isso demonstre que o ataque foi interrompido. Para avaliar a resposta, é

necessário verificar se a medida foi executada, quanto tempo demorou a produzir efeito e se provocou restrições indevidas ao tráfego legítimo. Esta distinção é especialmente importante em redes IoT, onde o isolamento incorreto de um dispositivo pode comprometer serviços que dependem da sua disponibilidade.

Alargando a análise aos 31 estudos incluídos, os mesmos padrões repetem-se (Tabelas XIII e XIV). Quanto ao local, em 12 estudos a resposta é aplicada por controladores SDN ou *switches* P4 [3]–[6], [20], [25], [27], [34], [40], [45], [48], [50], em 12 num *gateway* ou num nó *edge* ou *fog* [2], [15], [26], [33], [35], [36], [38], [39], [42], [44], [47], [49], e em três nos próprios dispositivos [12], [28], [37]; nos restantes quatro o local não é especificado. Quanto à medida, o descarte ou o bloqueio de tráfego é o mais comum, e 11 estudos chegam a isolar ou a pôr em quarentena o dispositivo inteiro [4], [6], [26]–[28], [33], [35], [36], [41], [42], [47]. A limitação de débito surge em seis como medida intermédia [3], [5], [27], [33], [38], [42], e só dois tornam o bloqueio temporário, com um temporizador aleatório [2] ou uma quarentena de 120 s [33]. Quanto ao desencadeamento, vários estudos exigem evidência acumulada ou uma segunda verificação antes de agir: um contador por IP que só bloqueia depois de três fluxos maliciosos [40], cinco previsões de ataque seguidas [34], limiares de confiança [5], [33], [36], a confirmação do alerta por pelo menos dois nós [42] ou limiares dinâmicos [28]. Seis recorrem a modelos de decisão, sobretudo aprendizagem por reforço [5], [25], [26], [37], [38], [44]. Em três estudos, pelo contrário, a resposta é proposta mas não chega a ser implementada [36], [38], [51].

Em suma, e respondendo à QF3, os mecanismos de resposta automática identificados na literatura consistem principalmente no descarte e bloqueio de tráfego, na aplicação de regras de *firewall*, no isolamento de nós ou dispositivos e na reconfiguração de políticas através de SDN. As ações são aplicadas em diferentes níveis, desde o próprio dispositivo IoT até aos *gateways* e equipamentos programáveis, podendo a decisão ser realizada localmente ou em componentes centralizados, incluindo a *cloud*. O desencadeamento depende de classificações produzidas por modelos de ML, de mecanismos adicionais de validação ou de modelos de decisão adaptativa. Apesar da diversidade de soluções, a revisão evidencia que a eficácia da contenção continua a ser avaliada de forma menos consistente do que a deteção, reforçando a importância de desenvolver e testar sistemas que integrem ambas as etapas.

E. Avaliação das soluções (QF4)

A literatura mostra uma evolução clara: muitos trabalhos começam com validações em ambiente controlado, *offline* e com *datasets* de referência, e evoluem para implementações em protótipos, *testbeds* e redes em tempo real. As abordagens de avaliação dividem-se, assim, em duas categorias.

Avaliação estática ou *offline*. É a etapa em que se calculam as métricas (exatidão, taxa de falsos positivos, matrizes de confusão) e se ajustam os parâmetros dos modelos, e em seis dos 31 estudos é a única. O MLAPS é avaliado apenas sobre o UNSW-NB15 [46]; Rawat *et al.* classificam fluxos do IoT-23 e, embora o resumo refira simulações e implementações reais, os resultados apresentados são apenas de classificação [51]. O mesmo acontece em mais quatro estudos [36], [38], [44], [50].

Avaliação em tempo real, protótipos e *testbeds*. A literatura orientada à aplicação prática valida os modelos em cenários simulados ou reais, com graus de realismo muito diferentes (Tabela XII). Sete estudos usam simulação [27], [28], [33], [37], [41], [43], [49], um deles complementado por um protótipo [27], e oito emulação de rede, quase sempre com Mininet [3]–[5], [20], [25], [40], [47], [48]. Tawfik *et al.*, por exemplo, correm o Mininet e o controlador Ryu num Raspberry Pi 3B, mas deixam a aprendizagem profunda para uma avaliação *offline* sobre o IoTID20 [48]. Dois usam um *testbed* virtualizado ou uma rede 5G em *software* [26], [34], e um corre em tempo real num servidor [39]. Só sete correm em *hardware* físico, quase sempre com Raspberry Pi [2], [6], [12], [15], [35], [42], [45]. O caso mais próximo de um dispositivo real é o de Kponyo *et al.*, em que quatro Raspberry Pi 3B fazem de dispositivos IoT e executam eles próprios a deteção e a resposta [12]. Mesmo nestes estudos, o treino e a avaliação dos modelos são normalmente feitos *offline*, e o protótipo serve para demonstrar a resposta.

O tempo até à contenção é medido em poucos estudos, e os resultados mostram onde está o estrangulamento. No caso do Slowloris, a mitigação demora 0,128 s, mas a deteção leva em média 20,6 s, porque um ataque lento só se revela ao fim de algum tempo [40]. Na rede 5G, a mitigação demora 2 a 4 s e a deteção entre 20 e 140 s, conforme o intervalo de recolha das estatísticas [34], e no ISAAF a deteção leva cerca de 17 s, contra 1 a 3 s do bloqueio [15]. Nestes casos, o tempo de deteção é entre cerca de 5 e 160 vezes maior do que o da resposta, o que justifica medir separadamente as duas parcelas de T_c (Eq. 13). Os tempos de milissegundos aparecem apenas em emulação ou simulação [4], [41]. O XAI-CPS mede a mitigação, também em simulação, com uma média de 47,8 s [43]. O estudo de Kponyo *et al.* é a exceção em *hardware* real, com deteção em 0,10 s e mitigação em 0,22 s [12], mas trata *floods*, que se revelam muito depressa.

O custo computacional é reportado de forma desigual. Os valores mais completos vêm de estudos que correm em equipamentos de baixo custo. Kponyo *et al.* medem a memória ocupada pelo mecanismo (21 a 28 MB, com um acréscimo de 2,82 pontos percentuais na ocupação de memória de um dos nós) e um consumo adicional de 35,8 mW, embora descrevam a CPU apenas de forma qualitativa [12]; no trabalho de Srilekha *et al.*, o cliente de *blockchain* acrescenta cerca de 15% de CPU e 50 MB de memória a um Raspberry Pi [42]; e no R-IDPS o agente instalado nos dispositivos acrescenta menos de meio ponto percentual de CPU e de memória [20]. Os restantes reportam tempos de inferência, a utilização de CPU dos controladores, entre 2% e 35% [3], [34], ou apenas tempos de treino, que no caso de modelos complexos treinados sobre milhões de registos podem ir de alguns minutos a quase duas horas [45]. Quatro estudos não reportam qualquer custo [25], [47], [48], [51].

Em síntese, a passagem da avaliação *offline* para protótipos em *hardware* real mostra que a deteção e a resposta podem correr em equipamentos de baixo custo com custos reduzidos [6], [12], [42]. Essa passagem, porém, só foi feita em sete dos 31 estudos, e quase sempre com um único tipo de ataque. É esta lacuna que a avaliação da solução proposta procura cobrir, ao medir T_c, a CPU e a memória do *gateway* nos mesmos cenários para os três modelos.

Tabela XIII
COMPARAÇÃO DOS TRABALHOS RELACIONADOS (S: SUPERVISIONADO; NS: NÃO SUPERVISIONADO; SS: SEMI-SUPERVISIONADO; R: POR REFORÇO)

Referência	*Dataset*(s)	Modelo(s)	Apr.	Onde atua a resposta	Resposta automática	Avaliação	Contenção medida	QA
Noetzold *et al.* [7] (2026)	CICIDS2017, CVE, DNS e texto (Kaggle)	BiLSTM, FFNN, CNN, RNN-CNN	S	–	Só alertas, numa interface em tempo real	*Offline*; inferência em servidor, Raspberry Pi 4 e ESP32-S3	–	–
Kponyo *et al.* [12] (2020)	Próprio (CPU e memória)	KNN sobre CPU e memória	S	Dispositivo (*host*)	Descarte por protocolo e porta (NetfilterQueue)	Tempo real (4 Raspberry Pi 3B)	Mitigação em 0,22 s	7,0
Ravi e Shalinie [2] (2020)	NSL-KDD, próprio	DFNN + K-means (SDRK)	SS	Nó *fog* (*switch*)	ACL de descarte do par MAC–IP com temporizador	*Testbed* real (30 Raspberry Pi)	Resposta $\geq$34% mais rápida que na *cloud*	6,5
Tawfik *et al.* [48] (2021)	IoTID20, próprio	RF, CNN, LSTM	S	Controlador SDN (Ryu)	Descarte ou bloqueio da porta ou do *host* (descrito)	Emulação (Mininet) e *offline*	–	6,0
Zolotukhin *et al.* [25] (2021)	CICIDS2018	PPO + MLP, CNN, LSTM, AE, SOM, BiGAN	S, NS, R	Controlador SDN	Descarte e redirecionamento de fluxos	Emulação	Curvas de recompensa	5,5
Mazhar *et al.* [20] (2022)	Próprio	SVM + assinaturas	S	Controlador SDN (ONOS)	Regras OpenFlow de descarte	Emulação (Mininet-WiFi)	– (atraso do tráfego legítimo: 8 s)	6,5
Rose *et al.* [26] (2022)	Próprio (público)	CNN + grafos de ataque (POMDP)	S	*Gateway* e *cloud*	Regras iptables: ligação específica ou isolamento do *host*	*Testbed* virtualizado	Cenários mitigados (sem tempo)	6,5
Saurabh *et al.* [46] (2022)	UNSW-NB15	DT + RF (2 níveis)	S	Não especificado	Descarte de pacotes, aplicado ao *dataset*	*Offline*	–	6,0
Andrade *et al.* [34] (2024)	CICDDoS2019	LR + NB	S	*Switch* P4 (ONOS)	IP do equipamento em tabela de descarte após 5 previsões	*Testbed* 5G	Mitigação em 2–4 s	6,5
El-Sayed *et al.* [3] (2024)	ToN-IoT, Edge-IIoTset, CICIoT2023	*Stacking* (RF, SVM, DT, kNN, … + MLP)	S	*Switches* P4 e controladores SDN	Descarte e limitação de débito	Emulação (Mininet-WiFi)	– (só deteção: 11–33 ms)	6,0
Garba *et al.* [45] (2024)	UNSW-NB15, CICDDoS2019, próprio	DT, KNN, LR, SVM	S	Controlador SDN (Ryu)	Descarte do MAC atacante (desencadeado pelo Snort)	*Hardware* real (Raspberry Pi)	–	6,0
Rawat *et al.* [51] (2024)	IoT-23	KNN, LR, LSTM	S	Não especificado	Contramedida proposta, não implementada	*Offline*	–	5,0
Toony *et al.* [4] (2024)	Edge-IIoTset, ToN-IoT, X-IIoTID	*Ensemble* ponderado de 6 classificadores (EWEA)	S	*Switches* P4 e controladores SDN	Descarte da origem, isolamento do nó e alerta aos outros controladores	Emulação (Mininet-WiFi)	Mitigação em $\approx$3 ms	7,5
Verma *et al.* [49] (2024)	KDDCUP'99, ToN-IoT, …, próprio	BayesNet, regressão logística	S	RSU (*edge* veicular)	Desvio (*sinkhole*) e descarte por assinatura	Simulação (NS-3) e *offline*	–	6,0
Al Abdulwahid [39] (2025)	CICIDS2017, CSE-CIC-IDS2018, UNSW-NB15	AE + RNN (CyberGuard-X)	SS	*Gateway* ou *edge*	Mitigação antes da cifra do *ransomware* (62%); mecanismo não descrito	Tempo real (i9, RTX 3090)	Latência 12,8 ms	7,0
Chaudhary *et al.* [50] (2025)	BoT-IoT	SVM, RF, J48, KNN, XGBoost	S	*Fog* com SDN	Bloqueio	*Offline*	–	5,5

Tabela XIV
COMPARAÇÃO DOS TRABALHOS RELACIONADOS (CONTINUAÇÃO)

Referência	*Dataset*(s)	Modelo(s)	Apr.	Onde atua a resposta	Resposta automática	Avaliação	Contenção medida	QA
Dange e Nitnaware [35] (2025)	BoT-IoT, CTU-13, próprio	LR, SVM, NB + regras	S	*Gateway* (Raspberry Pi)	Isolamento do dispositivo (não descrito)	*Testbed* real	–	5,5
Karam *et al.* [15] (2025)	MQTTset, próprio (público)	DT, GRU (escolhidos entre 15 modelos)	S	*Gateway* (Raspberry Pi 4)	Bloqueio do IP e da porta 1883 na *firewall* do *broker*, via REST	*Hardware* real e *offline*	Mitigação em 1–3 s (deteção ≈17 s)	7,5
Karibandi *et al.* [27] (2025)	Não identificado (11 classes)	XGBoost	S	*Cloud* e controlador SDN	Controlo de fluxos, segmentação e quarentena, por níveis	Simulação e protótipo	–	5,0
Kumar e Kumar [38] (2025)	CICIoT2023	DCSAtten-EBiGRU + Double DQN	S, R	*Gateway*	Bloquear, permitir ou limitar (sem resultados)	*Offline*	–	5,0
Premakumari *et al.* [37] (2025)	AWID, IoT-23, simulado	FNN + Q-learning e Double Q-learning	S, R	Dispositivo (nós sensores)	Ajuste do nível de encriptação (sem bloqueio)	Simulação (MATLAB)	Latência 120 ms	5,5
Ramirez-Martinez *et al.* [40] (2025)	CICIoT2023, IoT-23, próprio	KNN, DT, RF, LSTM, CNN, GRU, MLP	S	*Switch* P4	Descarte do IP de origem após 3 fluxos maliciosos	Emulação (Mininet, BMv2)	Mitigação em 0,128 s	7,0
Srilekha *et al.* [42] (2025)	UNSW-NB15, sintético	Extra Trees (*edge*), CNN-LSTM (*cloud*)	S	*Edge* e *cloud*, com *blockchain*	Bloqueio, limitação de débito e quarentena por *smart contract*	*Hardware* real (Raspberry Pi 4) e simulação	Resposta <3 s	7,0
Tariq e Ahanger [47] (2025)	IoT-23, MedBIoT, próprio	SAE + GRU	S, NS	*Gateway* ou *edge*	Isolamento, bloqueio de IP e regras de *firewall* por nível de risco	Emulação e *offline*	–	6,0
Arulkumar e K [44] (2026)	DAPT2020, KDDCUP'99, CICIoT2023, …	CNN-LSTM-BNN + XGBoost/LR; MDP multiagente	S, R	*Gateway* ou *edge*	Bloqueio e outras ações	*Offline*	– (latência 200 ms)	6,0
Gowtham *et al.* [5] (2026)	Edge-IIoTset, CIC-IDS2017, próprio	KPCA-TCN, Ghost-SE federado; Q-learning	S, R	Nós *fog* e controlador SDN	Etiquetar, limitar, descartar ou reencaminhar fluxos	*Testbed* Mininet SDN-*fog*	Latência de controlo 42–51 ms	6,5
Hunde e Ismail [28] (2026)	Próprio (Cooja)	DNN + limiares dinâmicos	S	Dispositivos (nós RPL)	*Blacklist* do nó atacante, propagada pela rede	Simulação (Cooja)	– (PDR +67%, latência −66%)	5,5
Hussain *et al.* [36] (2026)	NSL-KDD, MedBIoT, CIC-IDS2017, …	RF, LR, MLP (*trust score*)	S	*Gateway* e motor de política	*Allow*, *monitor*, *deny* ou quarentena (não implementadas)	*Offline* (Colab)	–	6,5
Ibrahim *et al.* [33] (2026)	IoTForge Pro, WUSTL-IIoT, X-IIoTID	CNN-BiLSTM	S	*Gateway* (não especificado)	Quarentena de 120 s ou limitação de débito	Simulação (*replay*)	Taxa de sucesso dos ataques 0–8,8%	6,0
Iqbal *et al.* [6] (2026)	WUSTL-IIoT-2021, X-IIoTID	RF + MLP (DeepGuard)	S	Controlador SDN	Bloqueio, quarentena em VLAN e contramedidas por domínio	*Hardware* real (Raspberry Pi 5) e *offline*	Latência <30 ms	7,0
Rasheed *et al.* [43] (2026)	SWaT, WADI, BATADAL, IIoT-Security	TAC-LSTM-GNN + ARLM	S	–	Bloqueio adaptado ao tipo de ataque	*Offline* e simulação	Mitigação média em 47,8 s (simulação)	6,5
Shukla e Agarwal [41] (2026)	UNSW-NB15, CICIDS2018, BoT-IoT	ST-GT (*graph transformer*)	S, SS	Não especificado	Controlo de acesso, reencaminhamento e isolamento de nós	Simulação (SUMO)	34,9 ms (simulação)	7,0
Este trabalho	IoT-23	RF, IF, AE	S, NS, SS	*Gateway* (nftables)	Quarentena do dispositivo após k de w fluxos, com expiração	Protótipo em contentores	T_c, CPU e memória (planeado)	–

F. Comparação com o estado da arte

A Tabela XIII compara os estudos incluídos segundo os critérios do formulário de extração, mais o trabalho de Noetzold *et al.* [7], que serviu de ponto de partida, e a solução proposta neste trabalho. As colunas correspondem às perguntas de pesquisa: modelos e tipo de aprendizagem (QF1), *datasets* (QF2), mecanismo de resposta e local onde atua (QF3), tipo de avaliação e medição da contenção (QF4).

Comparação com revisões anteriores. Antes de comparar os estudos, importa situar esta revisão face às que já existem. A Tabela XV compara-a com as quatro revisões usadas para afinar as perguntas de pesquisa (Secção III). Todas tratam a deteção e algumas referem a mitigação, mas nenhuma tem a resposta automática como objeto central, nem analisa onde é aplicada, como é desencadeada ou se chega a ser avaliada. Esta revisão faz das três questões as suas perguntas QF3 e QF4 e usa o EC6 para reter apenas estudos que abordam a resposta.

Tabela XV
COMPARAÇÃO DESTA REVISÃO COM REVISÕES ANTERIORES (N.I.: NÃO INDICADO NO RESUMO)

Revisão	Âmbito	Estudos	Resposta automática	Onde e como atua	Avaliação da resposta
Negera [8] (2022)	Deteção de *botnets* com ML em IoT com SDN	n.i.	Não é o foco	Não	Não
Vera-Mundaca [10] (2024)	Ameaças e mitigação em casas inteligentes	49 (2020–2024)	Parcial (estratégias)	Não	Não
Kandah [9] (2025)	Arquiteturas, ameaças e contramedidas em IoT	n.i.	Parcial (contramedidas)	Não	Não
Bhardwaj [32] (2025)	*Threat intelligence* e IDS (empresa, *cloud*, IoT)	n.i.	Parcial (tendência)	Não	Não
Esta revisão	Deteção e contenção de dispositivos IoT comprometidos com ML	31 (2020–2026)	Sim (EC6, QF3)	Sim (QF3)	Sim (QF4, T_c)

A leitura das Tabelas XIII e XIV, com os 31 estudos incluídos, mostra quatro padrões. O primeiro é que a resposta é aplicada sobretudo na infraestrutura de rede programável (*switches* P4 ou controladores SDN) [3]–[6], [20], [25], [27], [34], [40], [45], [48], [50], o que pressupõe equipamento que uma rede doméstica ou de um pequeno edifício raramente tem. O segundo é que poucos estudos medem a contenção em si: entre os que o fazem, os tempos vão de milissegundos [4], [41] e décimas de segundo [12], [40] a alguns segundos [15], [34], [42] e, em simulação, a dezenas de segundos [43]; os valores mais baixos são obtidos em emulação ou simulação. O terceiro é que, em 10 dos 31 estudos, a resposta não chega a ser avaliada numa rede: é apenas descrita, avaliada só *offline* sobre o *dataset* ou anunciada sem ser implementada [35], [36], [38], [44], [46]–[51]. O quarto é que só sete trabalhos foram testados em *hardware* real [2], [6], [12], [15], [35], [42], [45], e os que medem o tempo de contenção nesse *hardware* ou tratam apenas *floods* e decidem com base na CPU e na memória [12], ou levam alguns segundos a reagir [15], [42]. Nenhum dos estudos comparados junta, num *gateway* comum, a extração de fluxos, a comparação entre modelos supervisionados e não supervisionados e uma quarentena automática com o tempo de contenção medido. O mais próximo é o ISAAF, que corre num *gateway* Raspberry Pi em *hardware* real e mede o tempo de deteção e de bloqueio [15], mas usa apenas modelos supervisionados e bloqueia o IP e a porta do *broker* MQTT, em vez de isolar o dispositivo. É a combinação dos quatro elementos, e em particular a comparação entre famílias de modelos quanto ao efeito na contenção, que a solução proposta procura acrescentar.

V. SOLUÇÃO PROPOSTA

A solução que se propõe para a Parte 2 junta, num único equipamento, as duas funções que a revisão mostra estarem muitas vezes separadas: a deteção de dispositivos comprometidos a partir do tráfego e a sua contenção automática. Tudo corre no *gateway* da rede IoT, que é o único ponto por onde passa todo o tráfego dos dispositivos e onde é possível bloqueá-lo sem instalar nada nos equipamentos. A escolha segue a direção apontada por Noetzold *et al.* [7], que deixam a resposta automática e a execução na *edge* como trabalho futuro, e aproxima-se dos estudos que aplicam a contenção junto à origem do tráfego, num nó *fog* [2] ou num comutador de fronteira [40].

A. Arquitetura

O cenário de teste é montado com contentores Docker num único computador, o que permite repetir as experiências sem *hardware* dedicado e sem pôr em risco outras redes (Fig. 5). Há duas redes virtuais. Na rede IoT estão um ou dois contentores que fazem de dispositivos e um contentor atacante; na rede externa está um servidor que representa a Internet. Cada contentor da rede IoT fica numa sub-rede própria, ligada ao *gateway*; caso contrário, os dispositivos comunicariam diretamente pela *bridge* do Docker e o movimento lateral, como o varrimento da própria rede por um dispositivo infetado, não passaria pelo *gateway*. O *gateway* liga as redes, encaminha o tráfego entre elas e concentra quatro módulos:

1) **Captura e extração de fluxos.** O Zeek [19] observa as interfaces da rede IoT e escreve um registo por ligação (`conn.log`), com campos como a duração, os bytes e pacotes em cada sentido, o protocolo, o serviço, a porta de destino e o estado da ligação.
2) **Deteção.** Um serviço em Python lê os registos à medida que são escritos, normaliza as *features* e aplica os modelos treinados com scikit-learn [52].
3) **Decisão.** Agrega as previsões por dispositivo, isto é, por endereço IP de origem, e decide se o dispositivo deve ser isolado (Secção V-C).
4) **Contenção.** Acrescenta o endereço do dispositivo a um conjunto de quarentena no nftables [53], com um tempo de expiração. Uma regra na cadeia de encaminhamento descarta todo o tráfego cuja origem ou destino pertença

a esse conjunto, e cada decisão fica registada com a hora e o motivo.

Por razões éticas e de segurança, não é executado *malware* real. O tráfego malicioso vem das capturas do IoT-23 [11], que é reproduzido na rede IoT pelo contentor atacante com o `tcpreplay`, depois de os endereços serem reescritos com o `tcprewrite` para corresponderem aos dos contentores. Para o *gateway*, este tráfego é indistinguível do de um dispositivo infetado.

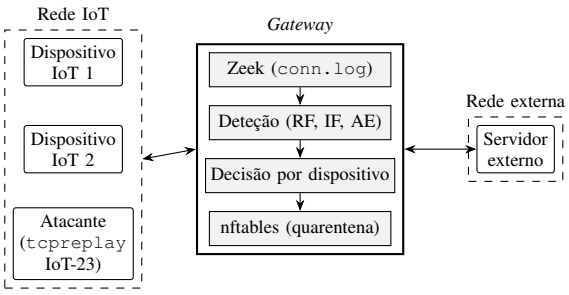


Figura 5. Arquitetura do sistema proposto, em contentores Docker (na rede IoT, cada dispositivo tem uma sub-rede própria).

Há três cuidados de implementação. O primeiro é que o Zeek só escreve uma ligação no `conn.log` quando esta termina ou expira, o que atrasaria a deteção de ligações longas, como as de C&C; por isso as *features* são também calculadas por intervalos durante a ligação. Estes registos parciais têm uma distribuição diferente da dos registos completos usados no treino, e essa diferença é avaliada na Parte 2. O segundo é a ordem das regras: a regra de quarentena fica antes da regra que aceita as ligações já estabelecidas, para que estas também sejam cortadas. O terceiro é a velocidade de reprodução das capturas, que é a original, porque uma velocidade diferente altera a duração e os intervalos entre pacotes, que são *features* dos modelos.

A escolha do IoT-23 tem uma vantagem prática importante: as capturas são distribuídas com os registos `conn.log` do Zeek já rotulados. Os modelos são, por isso, treinados com as mesmas *features* que o *gateway* calcula em funcionamento. Evita-se assim um problema visível em alguns dos estudos analisados, que treinam com as *features* de um *dataset* público e depois as tentam reproduzir de outra forma no protótipo [3].

B. Modelos de deteção

São comparados três modelos, um de cada família identificada na QF1. A ideia é avaliar o compromisso entre detetar bem os ataques conhecidos e conseguir sinalizar comportamentos que não apareceram no treino. As *features* numéricas de cada fluxo são normalizadas para $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{6}$$

e as categóricas (protocolo, serviço e estado da ligação) são codificadas com *one-hot encoding*.

A *Random Forest* [21] é o modelo supervisionado. É treinada com fluxos rotulados como benignos ou maliciosos e combina B árvores de decisão, cada uma treinada numa amostra *bootstrap* dos dados e com um subconjunto aleatório de *features* em cada divisão. A probabilidade de um fluxo ser malicioso é a fração de árvores que o classificam como tal:

$$p(x) = \frac{1}{B} \sum_{b=1}^{B} \mathbf{1}\left[h_b(x) = 1\right] \tag{7}$$

É um dos modelos mais frequentes na deteção de intrusões em IoT e serve de referência para os outros dois.

O *Isolation Forest* é o modelo não supervisionado. É treinado sem rótulos e classifica cada fluxo pelo *score* de anomalia da Eq. 1.

O *autoencoder*, implementado como uma rede MLP com uma camada central mais estreita do que a entrada (Eqs. 2 e 3), é o modelo semi-supervisionado. É treinado só com fluxos benignos, e um fluxo é considerado anómalo quando o erro de reconstrução da Eq. 4 ultrapassa o limiar θ. O limiar θ é fixado num percentil alto do erro observado no tráfego benigno de validação (por exemplo, o percentil 99), o que controla diretamente a taxa de falsos positivos esperada.

Para evitar resultados otimistas, a divisão entre treino e teste é feita por captura e não por fluxo: os fluxos de uma mesma captura do IoT-23 ficam todos do mesmo lado. Assim o modelo nunca é testado com tráfego quase idêntico ao que viu no treino. Os endereços IP, os identificadores das ligações e as marcas temporais são excluídos das *features*. A porta de destino não é usada diretamente: é agrupada em portas conhecidas, registadas e efémeras, porque no IoT-23 a porta 23 quase basta para identificar o tráfego malicioso e o modelo aprenderia esse atalho em vez do comportamento. Um ensaio de controlo sem a porta mostra se o resultado depende dela.

C. Decisão e contenção

Isolar um dispositivo por causa de um único fluxo suspeito seria arriscado, porque mesmo uma taxa de falsos positivos baixa, multiplicada pelos milhares de fluxos que um *gateway* vê por hora, levaria a quarentenas indevidas frequentes. Os estudos revistos que implementam a contenção resolvem isto exigindo várias evidências antes de agir: um bloqueio só depois de três fluxos maliciosos [40], ou de cinco previsões de ataque seguidas [34]. Segue-se a mesma abordagem. Para cada dispositivo i, considera-se a janela $W_i(t)$ com os seus últimos w fluxos, e o dispositivo é colocado em quarentena quando

$$D_i(t) = \mathbf{1}\left[\sum_{j \in W_i(t)} y_j \;\geq\; k\right], \qquad y_j \in \{0, 1\}, \tag{8}$$

em que $y_j = 1$ se o fluxo j foi sinalizado pelo modelo em uso. Os valores de w e de k são escolhidos na Parte 2, procurando o melhor equilíbrio entre o tempo até à contenção e o número de quarentenas indevidas.

A quarentena tem um tempo de expiração, tal como o bloqueio temporário usado por Ravi e Shalinie [2] e a quarentena de duração fixa de Ibrahim *et al.* [33]. Se o dispositivo

voltar a ser sinalizado depois de libertado, a quarentena é renovada. Isto reduz o custo de um falso positivo sem deixar um dispositivo realmente comprometido circular livremente. A regra bloqueia o dispositivo inteiro e não apenas a ligação suspeita, à semelhança da regra global de isolamento de *host* do IDERES [26], porque um equipamento comprometido pode mudar de destino ou de porta a qualquer momento.

D. Avaliação planeada

A avaliação responde às mesmas perguntas que foram usadas para analisar a literatura e decorre em três cenários:

- **S1, só tráfego benigno:** mede as quarentenas indevidas;
- **S2, ataque conhecido:** reprodução de capturas de famílias presentes no treino, como a Mirai;
- **S3, ataque não visto:** reprodução de capturas de uma família excluída do treino, para comparar a *Random Forest* com os modelos que só aprendem o comportamento normal. A família é escolhida entre as que não fazem varrimento de portas, como a Torii, porque muitas famílias do IoT-23 são variantes da Mirai com um comportamento quase igual, que a *Random Forest* detetaria na mesma.

Os três modelos são comparados no mesmo ponto de funcionamento, isto é, com o limiar de cada um ajustado para a mesma FPR, para que a comparação não dependa da escolha do limiar. Como referência sem ML, usa-se o detetor de varrimentos do próprio Zeek. Há ainda uma limitação conhecida: o IoT-23 tem poucas capturas de tráfego benigno, de apenas três dispositivos, pelo que os modelos que só aprendem o comportamento normal podem ter uma FPR alta no cenário S1 por encontrarem dispositivos que nunca viram, e não por causa de ataques.

A deteção é avaliada *offline* com as métricas que derivam da matriz de confusão [54], onde TP, FP, FN e TN são os verdadeiros positivos, falsos positivos, falsos negativos e verdadeiros negativos:

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN} \tag{9}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{10}$$

Num sistema que isola dispositivos, a taxa de falsos positivos é a que mede o custo de isolar um dispositivo legítimo, e por isso é reportada à parte:

$$FPR = \frac{FP}{FP+TN} \tag{11}$$

A estas junta-se a *Polygon Area Metric* (PAM) [55], também usada em [7]. A PAM representa n métricas normalizadas num gráfico radar e calcula a área do polígono que formam, dividida pela área máxima possível. A proposta original usa seis métricas (*accuracy*, sensibilidade, especificidade, AUC, índice de Jaccard e F1):

$$PAM = \frac{\frac{1}{2}\sin\left(\frac{2\pi}{n}\right)\sum_{i=1}^{n} r_i\, r_{i+1}}{\frac{n}{2}\sin\left(\frac{2\pi}{n}\right)} = \frac{1}{n}\sum_{i=1}^{n} r_i\, r_{i+1}, \tag{12}$$

com $r_{n+1} = r_1$. Um valor de 1 corresponde a um classificador perfeito em todas as métricas.

No protótipo, mede-se o tempo até à contenção, desde o primeiro pacote do ataque até à entrada do dispositivo na quarentena:

$$T_c = t_{\text{isolamento}} - t_{\text{ataque}} = T_{\text{deteção}} + T_{\text{resposta}} \tag{13}$$

Registam-se também a utilização de CPU e de memória do *gateway*, para perceber se a solução é viável num equipamento de *edge* com poucos recursos. Estes dados respondem, para a solução proposta, à mesma QF4 que foi colocada aos estudos revistos.

VI. Enquadramento na arquitetura Zero Trust

A solução da Secção V nasceu do problema dos dispositivos IoT comprometidos, mas as suas peças correspondem quase uma a uma aos componentes de uma arquitetura *Zero Trust* (ZTA). Esta secção faz esse enquadramento. Usa como referência o NIST SP 800-207 [56], o guia de implementação NIST SP 1800-35 [57] e o modelo de maturidade da CISA [58], e aplica-os a um cenário concreto: um edifício inteligente de um campus universitário, inserido numa cidade digital.

A. Porquê Zero Trust neste cenário

O modelo de segurança tradicional confia em tudo o que está dentro do perímetro. O problema, como nota o SP 800-207, é que depois de um atacante passar o perímetro o movimento lateral fica praticamente livre. A expressão *zero trust* foi popularizada por Kindervag [59] precisamente para inverter esse pressuposto: nenhum sujeito ou ativo é confiável só por estar na rede interna, e cada acesso é decidido por pedido, com o menor privilégio possível, como se a rede já estivesse comprometida [56]. A CISA leva a ideia mais longe e parte do princípio de que uma violação já ocorreu ou vai ocorrer [58].

O edifício do cenário tem câmaras IP, sensores ambientais, fechaduras e controladores de climatização e iluminação, muitos instalados e mantidos por empresas externas. É o caso que o SP 800-207 descreve para prestadores e visitantes (§4.3), com sistemas de climatização e iluminação geridos por terceiros. Hoje estes equipamentos partilham uma rede plana com o resto do edifício. Um único dispositivo infetado por uma variante da Mirai pode varrer a rede, tentar credenciais por omissão noutros equipamentos e participar em ataques DDoS, sem que nada o trave. O próprio SP 800-207 refere a Mirai quando discute a negação de serviço contra a infraestrutura de decisão (§5.2).

Há uma razão adicional para tratar estes dispositivos com *Zero Trust*, e não só com mais regras de *firewall*: os dispositivos IoT quase não têm identidade forte. He *et al.* [60] sublinham que estes equipamentos têm poucos atributos únicos e persistentes que permitam autenticá-los, e que os protocolos baseados em certificados são pesados para eles. O SP 800-207 lembra ainda que o endereço MAC não serve como identidade, porque pode ser falsificado (§3.4.1). Como não se pode confiar

na identidade declarada, a confiança tem de vir do que o dispositivo faz na rede.

Os fatores que levam o campus a adotar este modelo são de três tipos. Há a ameaça em si: *botnets* como a Mirai procuram ativamente equipamentos deste género. Há a operação: os prestadores externos precisam de aceder remotamente aos seus sistemas para manutenção, o que abre caminhos para dentro da rede. E há as obrigações legais e de serviço: as câmaras e as fechaduras tratam dados pessoais, abrangidos pelo RGPD, e uma falha nos controladores afeta o funcionamento do edifício. Os desafios também são conhecidos. O SP 1800-35 aponta a falta de inventário de ativos, a falta de visibilidade sobre os padrões de comunicação, a dificuldade de integrar tecnologias de maturidades diferentes e o receio de prejudicar as operações [57]. Neste cenário juntam-se mais dois: não é possível instalar agentes nos dispositivos, e isolar por engano uma fechadura ou uma câmara tem consequências físicas.

B. Princípios e componentes

O SP 800-207 define sete princípios (*tenets*, §2.1). A Tabela XVI mostra como cada um é tratado na solução proposta. Ela cobre bem os princípios ligados à monitorização contínua e à política dinâmica (4, 5 e 7). Os princípios ligados à autenticação e à cifra (2, 3 e 6) ficam fora do âmbito da Parte 2: dependem de capacidades que os dispositivos do cenário não têm.

Tabela XVI
PRINCÍPIOS DO NIST SP 800-207 NA SOLUÇÃO PROPOSTA (✓ COBERTO, ∼ PARCIAL, – FORA DO ÂMBITO)

Princípio (§2.1)		Na solução proposta
1. Todas as fontes de dados e serviços são recursos	✓	Cada dispositivo IoT é tratado como recurso com política própria
2. Comunicação protegida, qualquer que seja a localização	–	Fora do âmbito: muitos IoT não suportam cifra robusta
3. Acesso concedido por sessão	∼	Cada fluxo é avaliado; a quarentena corta também as ligações já abertas
4. Acesso decidido por política dinâmica	✓	A política muda com o comportamento observado (Eq. 8)
5. Monitorizar a integridade e a postura de todos os ativos	✓	Detetor ML sobre os fluxos de cada dispositivo
6. Autenticação e autorização dinâmicas antes do acesso	∼	Autorização revista em contínuo; sem autenticação forte
7. Recolher informação e usá-la para melhorar a política	✓	Registo de fluxos e decisões; retreino dos modelos

Os componentes lógicos de uma ZTA (§3.1) são três. O *Policy Engine* (PE) decide se um acesso é concedido, negado ou revogado. O *Policy Administrator* (PA) executa essa decisão, configurando o caminho entre o sujeito e o recurso. O *Policy Enforcement Point* (PEP) abre, monitoriza e fecha as ligações. PE e PA formam o *Policy Decision Point* (PDP). O PE apoia-se em várias fontes de informação, a que o SP 1800-35 chama *Policy Information Points* (PIP) [57]: o estado dos ativos, os registos de atividade da rede, a *threat intelligence* e a análise de segurança, entre outras.

A decisão do PE é tomada pelo algoritmo de confiança (§3.3). O SP 800-207 distingue algoritmos por critérios, em que todos os requisitos têm de ser cumpridos, de algoritmos por pontuação, que calculam um nível de confiança a partir das várias fontes e o comparam com um limiar. Distingue também algoritmos singulares, que avaliam cada pedido isoladamente, de algoritmos contextuais, que usam o histórico recente do sujeito e por isso detetam comportamentos atípicos. Em notação, um algoritmo por pontuação pode escrever-se como

$$C = \sum_{m} \alpha_m \, s_m, \qquad \text{acesso} \iff C < L_{\text{recurso}}, \tag{14}$$

em que s_m é o valor dado pela fonte m, α_m o peso que a organização lhe atribui e L_{recurso} o limiar aceite para o recurso. Aqui C é escrito como nível de suspeita, para coincidir com o sentido das saídas dos modelos.

C. A solução proposta como ZTA

A Tabela XVII e a Fig. 6 mostram a correspondência entre os módulos do *gateway* e os componentes do SP 800-207. Duas escolhas merecem justificação.

A primeira é a variante de implementação. O SP 800-207 descreve quatro (§3.2): agente e *gateway*, enclave, portal de recursos e *sandboxing*. A variante com agente pressupõe software instalado no dispositivo, o que é impossível numa câmara IP ou num controlador de climatização. Por isso a solução segue o modelo de enclave: um *gateway* na fronteira de um grupo de recursos que não consegue proteger-se sozinho. A abordagem é a microssegmentação (§3.1.2). Cada classe de dispositivos fica num segmento próprio, e todo o tráfego entre segmentos e para a Internet atravessa o PEP.

A segunda é o algoritmo de confiança. O detetor é um algoritmo contextual por pontuação. O contexto é o histórico recente de fluxos de cada dispositivo, e a pontuação é a saída dos modelos: a probabilidade da *Random Forest* (Eq. 7), o *score* do *Isolation Forest* (Eq. 1) ou o erro do *Autoencoder* (Eq. 4). Na versão da Parte 2 há uma só fonte de informação. A Eq. 14 reduz-se então a $C_i(t) = \frac{1}{w}\sum_{j \in W_i(t)} y_j$, e, com $L_{\text{recurso}} = k/w$, o acesso é negado e a quarentena imposta quando $C_i(t) \geq k/w$, o que é exatamente a regra da Eq. 8. A formulação geral deixa espaço para acrescentar outras fontes com pesos próprios, como listas de servidores de C&C conhecidos ou o inventário de dispositivos.

Tabela XVII
COMPONENTES DO SP 800-207 E MÓDULOS DA SOLUÇÃO

Componente	Na solução proposta
Sujeitos e recursos	Dispositivos IoT do edifício (identidades de máquina) e serviços que contactam
PIP / fontes de dados	Zeek (registos de fluxos), detetor ML, inventário de dispositivos; *threat intelligence* como extensão
PE	Módulo de decisão por dispositivo: algoritmo contextual por pontuação (Eq. 8)
PA	Serviço que traduz a decisão numa operação sobre o conjunto de quarentena
PEP	*Gateway* com nftables, modelo de enclave por segmento IoT
Plano de controlo	Comunicação local entre detetor, decisão e nftables, separada do tráfego dos dispositivos

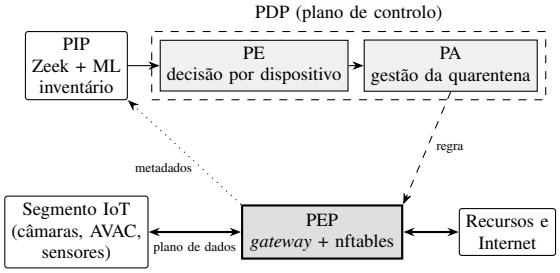


Figura 6. A solução proposta segundo os componentes lógicos do NIST SP 800-207.

A Tabela XVIII descreve os casos de uso, alinhados com os do SP 1800-35 [57]. O guia do NCCoE demonstra, entre outros, a descoberta de ativos (caso A) e a revisão contínua da confiança durante a sessão (casos F). Nestes últimos, um *endpoint* que deixa de estar conforme perde o acesso, recupera-o quando volta à conformidade, e um *endpoint* suspeito de estar comprometido vê os seus pedidos negados e as sessões terminadas. A diferença é que, no guia, a conformidade é verificada por agentes instalados nos equipamentos; aqui é inferida do tráfego.

Tabela XVIII
CASOS DE USO DA SOLUÇÃO

	Situação e resposta	SP 1800-35
1	Um dispositivo novo liga-se: é registado no inventário e o seu tráfego passa a ser observado	A (descoberta)
2	Um dispositivo começa a varrer a rede ou a contactar um C&C: é colocado em quarentena e todas as ligações são cortadas	F-14 a F-17
3	A quarentena expira e o tráfego volta ao normal: o dispositivo recupera o acesso; se reincidir, a quarentena é renovada	F-4 e F-5
4	Um dispositivo crítico é isolado por engano: o administrador liberta-o manualmente e o caso entra no re-treino	–

D. Maturidade e plano de transição

O modelo da CISA [58] organiza a ZTA em cinco pilares (identidade, dispositivos, redes, aplicações e dados), três capacidades transversais (visibilidade e análise, automação e orquestração, governação) e quatro níveis de maturidade (tradicional, inicial, avançado e ótimo). Nos pilares que importam aqui, dispositivos e redes, o edifício parte do nível tradicional: rede plana, inventário manual e nenhuma resposta automática. A solução proposta leva a visibilidade e a automação ao nível avançado, que a CISA associa à deteção automática de anomalias na rede e ao isolamento automático de dispositivos. O nível ótimo, que prevê também a remediação automática, fica fora do âmbito. O protótipo da Parte 2 entrega a deteção e a quarentena automáticas; o inventário a partir dos fluxos e a microssegmentação por classe de dispositivo pertencem ao cenário do edifício e ao plano de transição descrito a seguir. Ali *et al.* [61] resumem bem este passo ao afirmar que a função central da automação é converter a deteção de ameaças em prevenção.

A Tabela XIX resume as lacunas da situação de partida e o nível que a solução permite atingir em cada função relevante. O pilar da identidade fica limitado ao acesso dos administradores ao PDP, que deve exigir autenticação multifator. Para os dispositivos, como se viu, a identidade forte não é realista.

Tabela XIX
LACUNAS DA SITUAÇÃO DE PARTIDA E NÍVEL ATINGIDO (CISA ZTMM V2)

Pilar / função	Situação de partida	Com a solução proposta
Dispositivos: visibilidade e análise	Inventário manual, sem deteção (tradicional)	Deteção automática de anomalias (avançado); inventário a partir dos fluxos no plano de transição
Dispositivos: automação	Isolamento manual, quando existe (tradicional)	Quarentena automática com expiração e renovação (avançado)
Redes: segmentação	Rede plana (tradicional)	Microssegmentação por classe, com o *gateway* como PEP (avançado, no plano de transição; o protótipo isola cada dispositivo)
Redes: visibilidade e análise	Monitorização só na fronteira (tradicional)	Deteção de anomalias com ML em todo o tráfego IoT (avançado)
Identidade	Contas locais sem MFA (tradicional)	MFA no acesso dos administradores ao PDP (inicial)

A transição segue os passos de migração do SP 800-207 (§7.3) e a sequência proposta pelo SP 1800-35:

1) inventariar os dispositivos e os fluxos de que dependem, com o próprio Zeek;
2) segmentar a rede por classe de dispositivo, com o *gateway* como único caminho entre segmentos;
3) pôr o detetor em modo só de relatório, como recomenda o §7.3.6, para conhecer o comportamento normal e afinar w e k sem afetar o serviço;
4) ativar a quarentena automática, primeiro nos segmentos menos críticos;
5) rever periodicamente os modelos e alargar as fontes de informação do PE.

E. Riscos da própria arquitetura

Uma ZTA também introduz riscos, e o SP 800-207 dedica-lhes uma secção (§5). Três são diretamente relevantes. O primeiro é a subversão do processo de decisão (§5.1): quem controlar o detetor ou as regras pode isolar dispositivos legítimos ou libertar os comprometidos. Mitiga-se com um plano de controlo isolado, acesso dos administradores com autenticação multifator e registo de todas as decisões. O segundo é a negação de serviço contra o PEP (§5.2): se o *gateway* cair, o edifício perde a ligação, pelo que em produção seria necessária redundância. O terceiro é o risco das entidades não pessoais que tomam decisões, como modelos de ML (§5.7), cujo efeito mais visível são os falsos positivos. A janela de w fluxos, a quarentena com expiração e o período inicial só de relatório respondem diretamente a este risco.

Há ainda uma questão de privacidade. Monitorizar o tráfego de um edifício pode tocar em dados pessoais. Por isso a solução usa apenas metadados de fluxos, sem conteúdo, seguindo o princípio da minimização do RGPD. O próprio SP 800-207 reconhece, no §5.4, que técnicas de ML aplicadas a metadados permitem analisar tráfego que não pode ser decifrado.

F. Lacunas e contributo

As referências oficiais deixam o caso IoT em aberto. O SP 1800-35 exclui explicitamente do seu âmbito os sistemas de controlo industrial, a tecnologia operacional e os dispositivos IoT [57]. O modelo da CISA declara que não trata os desafios de algumas classes de dispositivos IoT nem a incorporação de ML e IA em soluções *Zero Trust* [58]. Na literatura científica, Syed *et al.* [62] apontam o cálculo da confiança e do risco como um dos obstáculos a uma ZTA completa. He *et al.* [60] apresentam o ML como o método de avaliação de confiança mais preciso, mas também o mais exigente em computação. Li *et al.* [63] propõem uma arquitetura *Zero Trust* para IoT industrial centrada na autenticação dos dispositivos, com *blockchain*.

Do lado da aplicação, o FURZE de Vanickis *et al.* [64] é o trabalho mais próximo: gera e retira regras de *firewall* entre zonas à medida que o risco de uma sessão muda. O risco, porém, é calculado com lógica difusa sobre atributos, e não a partir do tráfego. O eZTrust de Zaheer *et al.* [65] mostra que até entre microsserviços o endereço IP não é uma identidade fiável, mas exige controlar o *kernel* do emissor, o que não é possível num dispositivo IoT. A abordagem MUD [66], [67] é complementar: o fabricante declara o comportamento de rede esperado do dispositivo, e o *gateway* aplica essa lista. Depende, contudo, de os fabricantes publicarem estes perfis.

A solução proposta ocupa o espaço entre estes trabalhos. A confiança é inferida do comportamento observado, sem exigir nada ao dispositivo nem ao fabricante. A inferência corre no *gateway*, o que contorna a falta de recursos dos dispositivos. E a resposta é aplicada automaticamente pelo PEP. Isto dá aos dispositivos IoT a verificação periódica da saúde dos recursos que o SP 1800-35 considera muito desejável e que vários dos seus *builds* não conseguiram assegurar [57].

VII. Discussão

A. Lacunas e oportunidades

Da comparação resultam cinco lacunas, resumidas na Tabela XX com a forma como a solução proposta lhes responde.

A primeira é que a contenção é anunciada com mais frequência do que é avaliada. Mesmo entre os estudos incluídos, que já passaram um critério de exclusão sobre a resposta (EC6), vários descrevem ou anunciam um mecanismo que depois não avaliam numa rede [35], [36], [38], [46], [48], [51], e só oito medem quanto tempo demora a conter um ataque [4], [12], [15], [34], [40]–[43]. A segunda é a dependência de infraestrutura programável. Boa parte das propostas pressupõe *switches* P4 ou um controlador SDN, que não existem numa casa, numa pequena empresa ou num edifício com uma rede convencional, onde o ponto de controlo natural é um *gateway* Linux.

A terceira diz respeito aos dados. Os *datasets* estão muito dispersos, vários não são de IoT, e o tráfego próprio quase nunca é publicado, o que impede comparar resultados entre estudos. A quarta é a escassa atenção aos falsos positivos: poucos estudos reportam a FPR, embora seja a métrica que determina se a contenção automática é aceitável. A resposta que a literatura encontra na prática, exigir evidência acumulada antes de agir [34], [40], raramente é discutida como tal. A quinta é que nenhum estudo compara de forma sistemática modelos supervisionados e não supervisionados quanto ao efeito na contenção, embora a escolha entre os dois determine se o sistema reage a ataques que não estavam no treino.

A estas juntam-se as lacunas das referências oficiais de *Zero Trust*, que deixam o IoT e o ML fora do seu âmbito [57], [58], como se discutiu na Secção VI.

Tabela XX
Lacunas identificadas e resposta da solução proposta

Lacuna	Resposta na solução proposta
Contenção descrita mas pouco avaliada	Medição de T_c, separando deteção e resposta (Eq. 13)
Dependência de P4 ou SDN	Contenção num *gateway* comum, com nftables
Datasets dispersos e não IoT	IoT-23, com os mesmos registos Zeek no treino e no protótipo
FPR pouco reportada	FPR, quarentenas indevidas (cenário S1) e decisão por k de w fluxos (Eq. 8)
Falta de comparação entre famílias de modelos	RF, IF e AE avaliados nos mesmos cenários, incluindo ataques não vistos (S3)
IoT e ML fora das referências de *Zero Trust*	Enquadramento no NIST SP 800-207 e na CISA (Secção VI)

B. Ameaças à validade

Como em qualquer revisão sistemática, os resultados dependem das escolhas do protocolo. Identificam-se quatro ameaças.

Pesquisa. A *string* foi aplicada apenas ao resumo e exige termos ligados à resposta. Estudos que só descrevem a contenção no texto completo podem ter ficado de fora, e a *string* não inclui termos como *containment*, *blocking* ou *isolation*: o

último foi retirado da primeira versão por devolver demasiados resultados fora do tema, e os outros dois podem ter afastado estudos que usam essas palavras em vez de *mitigation* ou *prevention*. No ScienceDirect foi usada uma versão reduzida da *string*, a pesquisa limitou-se a cinco bases, a artigos em inglês e ao período 2020–2026, e 17 registos não foram exportados ou importados para o Parsifal. Também não foi feita pesquisa por *snowballing*, isto é, pelas referências e pelas citações dos estudos incluídos, que poderia ter encontrado estudos relevantes fora das cinco bases.

Seleção. A triagem foi dividida por blocos, cada um com um revisor. Para reduzir a subjetividade, os critérios foram escritos antes da triagem, os casos duvidosos passaram sempre à etapa seguinte e as decisões ficaram registadas no Parsifal. O critério EC6 foi introduzido depois da primeira leitura dos resumos. Embora seja um refinamento previsto no protocolo e esteja documentado, pode ter afastado estudos com contributos parciais sobre a resposta. Como cada bloco aplicou os critérios de forma independente, o grau de exigência com que o EC6 foi lido também variou: alguns estudos foram incluídos por proporem uma resposta, mesmo sem a avaliarem. Essa diferença fica visível na Tabela XIII e foi tida em conta na leitura dos resultados. Na revisão final do conjunto, seis estudos que tinham passado a avaliação de qualidade foram excluídos pelo EC6, por não proporem qualquer mecanismo de resposta: limitavam-se à classificação, a um alerta ou a uma notificação ao utilizador. Cinco estudos foram triados por dois revisores, por aparecerem em dois blocos: em quatro as decisões e as pontuações coincidiram, e no quinto [27] um revisor excluiu-o (4,5) e outro incluiu-o (5,0), tendo prevalecido a inclusão. Este caso mostra a sensibilidade ao limiar: meio ponto numa pergunta decide se um estudo entra. Pela mesma razão, a QA5 não foi tornada eliminatória: o EC6 já exige que o estudo proponha uma resposta, e a QA5 serve para distinguir os que a avaliam dos que só a descrevem. Por isso há estudos incluídos com QA5 negativa, visíveis na Tabela VIII.

Extração e qualidade. A avaliação de qualidade e a extração foram feitas por um revisor por estudo. Em alguns artigos as tabelas e figuras não eram legíveis no texto disponível, e vários contêm incoerências numéricas, como se referiu na Secção IV. Os valores apresentados são os reportados pelos autores e não foram reproduzidos.

Generalização. O número de estudos incluídos é reduzido, pelo que as proporções indicadas descrevem este conjunto e não devem ser lidas como estimativas da área no seu todo.

VIII. Conclusões e trabalhos futuros

Esta revisão procurou saber como é que a análise de tráfego com ML é usada para detetar e conter automaticamente dispositivos IoT comprometidos (QG). A resposta curta é que a deteção está madura, mas a contenção ainda não. Dos 491 estudos únicos devolvidos pela pesquisa, 31 cumpriram os critérios de seleção e a avaliação de qualidade. A maioria deteta os ataques com uma exatidão acima de 97%, mas só 13 descrevem e avaliam o mecanismo de resposta, e só oito medem quanto tempo demora a conter um ataque.

Quanto aos modelos e às *features* (QF1), domina a aprendizagem supervisionada: a RF, as árvores de decisão, o KNN e a SVM aparecem ao lado de redes CNN, LSTM e GRU, muitas vezes combinadas em *ensembles*. As abordagens não supervisionadas e semi-supervisionadas são poucas e, mesmo quando surgem ao lado de modelos supervisionados, nenhum estudo mede o efeito de cada família na contenção. As *features* são, na grande maioria, estatísticas de fluxo, o que permite trabalhar com tráfego cifrado e sem tocar nos dispositivos.

Quanto aos dados (QF2), os 31 estudos usam 29 *datasets* públicos diferentes, 13 recorrem a pelo menos um que não é de IoT, e os dados próprios raramente são publicados. O DoS/DDoS aparece em 28 estudos e é quase sempre o ataque contra o qual a resposta é demonstrada. A FPR, que indica quantos dispositivos legítimos seriam isolados por engano, só tem valor explícito em 13 estudos, e apenas dez medem a eficácia da própria resposta.

Quanto à resposta (QF3), o descarte ou o bloqueio de tráfego é a medida mais comum, e 11 estudos isolam o dispositivo inteiro. A resposta é aplicada sobretudo em controladores SDN ou *switches* P4 (12 estudos) e em *gateways* ou nós *edge* (12). Os estudos mais cuidadosos não reagem a um único alerta: exigem evidência acumulada ou usam modelos de decisão, como a aprendizagem por reforço. Em três, pelo contrário, a resposta é proposta mas não chega a ser implementada.

Quanto à avaliação (QF4), seis estudos são avaliados apenas *offline* e só sete correm em *hardware* real. Nos estudos com ataques lentos ou com recolha periódica de estatísticas, a deteção é a parcela dominante: 17 a 140 s, contra 0,13 a 4 s da resposta [15], [34], [40]. Nos *floods*, as duas parcelas ficam abaixo do segundo [4], [12]. O custo computacional é reportado de forma desigual e raramente no equipamento onde a deteção corre.

A comparação das Tabelas XIII e XIV mostra a lacuna que motiva este trabalho: nenhum estudo junta, num *gateway* comum, a extração de fluxos, a comparação entre modelos supervisionados e não supervisionados e uma quarentena automática com o tempo de contenção medido. A solução proposta (Secção V) ocupa esse espaço com um *gateway* em contentores que extrai fluxos com o Zeek, compara a RF, o *Isolation Forest* e o *autoencoder* nos mesmos cenários e isola com nftables os dispositivos com k fluxos maliciosos numa janela de w. Enquadrada no NIST SP 800-207 (Secção VI), a solução faz do *gateway* um PEP e do detetor um algoritmo de confiança contextual, e cobre um caso que as referências oficiais de *Zero Trust* deixam em aberto: o de dispositivos que não conseguem autenticar-se nem correr agentes de segurança.

Trabalhos futuros. O passo imediato é a Parte 2: implementar o protótipo e avaliá-lo nos cenários S1 a S3, com as métricas das Eqs. (9)–(11), a PAM (Eq. 12) e o tempo de contenção T_c (Eq. 13), separado em deteção e resposta, a par da CPU e da memória do *gateway*. Os resultados dirão se os modelos que só aprendem o comportamento normal compensam, perante ataques não vistos no treino, a menor precisão que costumam ter face à RF. Depois disso, há quatro extensões naturais: uma resposta graduada, que limite o débito

antes de isolar e só isole quando a evidência persiste; mais fontes de informação para o PE, como os perfis MUD dos fabricantes e o inventário de dispositivos; a validação com outros *datasets* e em *hardware* real, com um Raspberry Pi a fazer de *gateway*; e a atualização contínua dos modelos, com aprendizagem incremental, para acompanhar a mudança do tráfego legítimo ao longo do tempo.

REFERÊNCIAS

[1] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1093–1110.

[2] N. Ravi and S. M. Shalinie, "Semisupervised-learning-based security to detect and mitigate intrusions in IoT network," *IEEE Internet of Things Journal*, vol. 7, no. 11, pp. 11 041–11 052, 2020.

[3] A. El-Sayed, W. Said, A. Tolba, Y. Alginahi, and A. A. Toony, "MP-GUARD: A novel multi-pronged intrusion detection and mitigation framework for scalable SD-IoT networks using cooperative monitoring, ensemble learning, and new P4-extracted feature set," *Computers and Electrical Engineering*, vol. 118, p. 109484, 2024.

[4] A. A. Toony, F. Alqahtani, Y. Alginahi, and W. Said, "MULTI-BLOCK: a novel ML-based intrusion detection framework for SDN-enabled IoT networks using new pyramidal structure," *Internet of Things*, vol. 26, p. 101231, 2024.

[5] M. S. Gowtham, K. S. Bhuvaneshwari, M. Ramkumar, and R. S. Kumar, "FedMP-LDL: Lightweight DDoS detection and mitigation in SDN-fog IoT networks," *Internet of Things*, vol. 38, p. 101996, 2026.

[6] A. Iqbal, S. Asghar, and M. I. Tamimy, "A multilayered deep learning framework for cyber attack detection and mitigation in a heterogeneous IIoT ecosystem," *Journal of Information Security and Applications*, vol. 96, p. 104301, 2026.

[7] D. Noetzold, V. R. Q. Leithardt, J. L. V. Barbosa, and A. G. d. M. Rossetto, "Anomaly detection monitoring: A proactive approach with deep learning integration," *Revista Brasileira de Computação Aplicada*, vol. 18, no. 2, pp. 32–48, 2026.

[8] W. G. Negera, F. Schwenker, T. G. Debelee, H. M. Melaku, and Y. M. Ayano, "Review of botnet attack detection in SDN-enabled IoT using machine learning," *Sensors*, vol. 22, no. 24, p. 9837, 2022.

[9] F. Kandah, T. Mendis, L. Medury, H. Sherawat, and H. Wang, "Navigating IoT security: Architectures, emerging threats, and adaptive countermeasures," *IEEE Access*, vol. 13, pp. 98 888–98 908, 2025.

[10] V. G. Vera-Mundaca and J. Y. Aquino-Trujillo, "Cybersecurity strategies for smart homes: A systematic review of threats and mitigation approaches in IoT environments," in *Proc. LACCEI International Multi-Conference for Engineering, Education and Technology*, 2024.

[11] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Zenodo, Version 1.0.0, 2020.

[12] J. J. Kponyo, J. O. Agyemang, G. S. Klogo, and J. O. Boateng, "Lightweight and host-based denial of service (DoS) detection and defense mechanism for resource-constrained IoT devices," *Internet of Things*, vol. 12, p. 100319, 2020.

[13] OASIS, "MQTT version 5.0," OASIS Standard, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[14] Z. Shelby, K. Hartke, and C. Bormann, "The constrained application protocol (CoAP)," RFC 7252, IETF, 2014.

[15] K. Karam, A. Aqachtoul, A. Elamrani, M. Najib, N. Rafalia, I. Moumen, J. Abouchabaka, and M. Bakhouya, "ISAAF: an IoT security and attack prevention framework using AI-driven predictive analytics," *Scientific Reports*, vol. 15, p. 44913, 2025.

[16] B. Claise, B. Trammell, and P. Aitken, "Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information," RFC 7011, IETF, 2013.

[17] B. Claise, "Cisco systems NetFlow services export version 9," RFC 3954, IETF, 2004.

[18] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[19] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[20] N. Mazhar, R. Saleh, R. Zaba, M. Zeeshan, M. M. Hameed, and N. Khan, "R-IDPS: Real time SDN-based IDPS system for IoT security," *Computers, Materials & Continua*, vol. 73, no. 2, pp. 3099–3118, 2022.

[21] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[22] R. Iranzad and X. Liu, "A review of random forest-based feature selection methods for data science education and applications," *International Journal of Data Science and Analytics*, vol. 20, no. 2, pp. 197–211, Aug 2025.

[23] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.

[24] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

[25] M. Zolotukhin, T. Hämäläinen, and R. Immonen, "Curious SDN for network attack mitigation," in *Proc. IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, 2021, pp. 630–635.

[26] J. R. Rose, M. Swann, K. P. Grammatikakis, I. Koufos, G. Bendiab, S. Shiaeles, and N. Kolokotronis, "IDERES: Intrusion detection and response system using machine learning and attack graphs," *Journal of Systems Architecture*, vol. 131, p. 102722, 2022.

[27] V. S. L. H. Karibandi, V. Jyothi, S. R. Miriyala, R. S. Mallisetti, C. V. K. Kanchipati, and R. Kollabathula, "CyberDome: Cloud based intrusion detection and mitigation," in *Proc. 3rd International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*, 2025.

[28] E. T. Hunde and S. Ismail, "RankShield: A hybrid deep learning and threshold-based framework for the detection and mitigation of RPL rank attacks in IoT networks," *Computer Networks*, vol. 285, p. 112375, 2026.

[29] S. Tan, W. Liu, Q. Dong, S. Chan, S. Yu, X. Zhong, and D. He, "Hitting moving targets: Intelligent prevention of IoT intrusions on the fly," *IEEE Internet of Things Journal*, vol. 10, no. 23, pp. 21 000–21 012, 2023.

[30] S. Keshav, "How to read a paper," *ACM SIGCOMM Computer Communication Review*, vol. 37, no. 3, pp. 83–84, 2007.

[31] Grupo SRSI, "Repositório do projeto: deteção e contenção de dispositivos IoT comprometidos," https://github.com/afonsolopes41/srsi-iot-ml-containment, 2026.

[32] A. Bhardwaj, S. K. Srivastava, P. Sagar, and M. K. Mahto, "A systematic review of threat intelligence and intrusion detection systems in cybersecurity architecture," in *Proc. 1st IEEE Uttar Pradesh Section Women in Engineering International Conference on Electrical, Electronics and Computer Engineering (UPWIECON)*, 2025, pp. 787–792.

[33] H. Ibrahim, L. A. C. Ahakonye, J.-M. Lee, and D.-S. Kim, "A unified AI-PureChain framework for verifiable intrusion prevention in industrial IoT systems," *IEEE Internet of Things Journal*, vol. 13, no. 7, pp. 12 906–12 919, 2026.

[34] D. P. Andrade, K. Akkaya, A. Perez-Pons, S. Uluagac, and A. Sahin, "DDoS attack detection and mitigation in 5G networks using P4 and SDN," in *Proc. IEEE 49th Conference on Local Computer Networks (LCN)*, 2024, pp. 1–9.

[35] S. Dange and P. Nitnaware, "A novel machine learning and internet of things (IoT) based dual monitoring system for proactive botnet attack prevention," *SN Computer Science*, vol. 6, no. 6, p. 708, 2025.

[36] H. Hussain, S. Mishra, and R. Alshenaifi, "Post-quantum cognitive zero trust architecture for healthcare IoT devices," *PLOS ONE*, vol. 21, no. 5, p. e0348600, 2026.

[37] S. B. N. Premakumari, G. Sundaram, M. Rivera, P. Wheeler, and R. E. Pérez Guzmán, "Reinforcement Q-learning-based adaptive encryption model for cyberthreat mitigation in wireless sensor networks," *Sensors*, vol. 25, p. 2056, 2025.

[38] K. P. Kumar and N. S. Kumar, "Advanced attack mitigation in IoT gateway protocols," *Computers & Security*, vol. 157, p. 104539, 2025.

[39] A. Al Abdulwahid, "AI-Driven identification of attack precursors: A machine learning approach to predictive cybersecurity," *Computers, Materials and Continua*, vol. 85, no. 1, pp. 1751–1777, 2025.

[40] E. D. Ramirez-Martinez, J. A. Pérez-Díaz, and N. M. Yungaicela-Naula, "P4-assisted Slowloris DDoS attack detection in IoT environments by using ML and DL," *Computer Networks*, vol. 267, p. 111364, 2025.

[41] P. K. Shukla and R. Agarwal, "A deep learning-driven cyber attack avoidance framework for secure IoT-enabled smart city transportation systems," *The Journal of Supercomputing*, vol. 82, p. 186, 2026.

[42] T. Srilekha, S. Komakula, and H. Manthena, "A hybrid intelligence-driven model for enhancing security and trust in integrated edge to cloud networks through real-time anomaly detection and tamper-proof storage," in *Proc. 2nd Int. Conf. Recent Trends in Electrical, Electronics and Computer Technologies (ICRTEECT)*. IEEE, 2025.

[43] I. Rasheed, H. Mostafa, and G. Mohayudin, "XAI-CPS: An explainable hybrid deep learning framework for real-time anomaly detection and adaptive threat mitigation in networked Cyber–Physical systems," *Computer Networks*, vol. 283, p. 112320, 2026.

[44] D. Arulkumar and K. K, "Metastack-aptnet: An ensemble deep learning framework for advanced persistent threat detection and mitigation in cyber-physical systems using blockchain technology," *Computers and Electrical Engineering*, vol. 130, p. 110838, 2026.

[45] U. H. Garba, A. N. Toosi, M. F. Pasha, and S. Khan, "SDN-based detection and mitigation of DDoS attacks on smart homes," *Computer Communications*, vol. 221, pp. 29–41, 2024.

[46] K. Saurabh, S. Singh, R. Vyas, O. P. Vyas, and R. Khondoker, "MLAPS: A machine learning based second line of defense for attack prevention in IoT network," in *Proc. IEEE 19th India Council International Conference (INDICON)*, 2022.

[47] U. Tariq and T. A. Ahanger, "Employing SAE-GRU deep learning for scalable botnet detection in smart city infrastructure," *PeerJ Computer Science*, vol. 11, p. e2869, 2025.

[48] M. Tawfik, N. M. Al-Zidi, B. Alsellami, A. M. Al-Hejri, and S. Nimbhore, "Internet of things-based middleware against cyber-attacks on smart homes using software-defined networking and deep learning," in *Proc. 2nd International Conference on Computational Methods in Science and Technology (ICCMST)*, 2021, pp. 7–13.

[49] A. Verma, R. Saha, G. Kumar, and M. Conti, "PETRAK: A solution against DDoS attacks in vehicular networks," *Computer Communications*, vol. 221, pp. 142–154, 2024.

[50] P. Chaudhary, A. Singh, and B. Gupta, "Dynamic multiphase DDoS attack identification and mitigation framework to secure SDN-based fog-empowered consumer IoT networks," *Computers and Electrical Engineering*, vol. 123, p. 110226, 2025.

[51] R. S. Rawat, M. Diwakar, U. Garg, and P. Srivastava, "An intelligent defensive and countermeasure mechanism for IoT botnet threats," in *Proc. International Conference on Artificial Intelligence and Emerging Technology (Global AI Summit)*, 2024, pp. 254–259.

[52] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[53] The Netfilter Project, "nftables," https://netfilter.org/projects/nftables/, 2026, acedido em outubro de 2026.

[54] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.

[55] O. Aydemir, "A new performance evaluation metric for classifiers: Polygon area metric," *Journal of Classification*, vol. 38, pp. 16–26, 2021.

[56] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology, NIST Special Publication 800-207, Aug. 2020.

[57] National Cybersecurity Center of Excellence, "Implementing a zero trust architecture: High-level document," National Institute of Standards and Technology, NIST Special Publication 1800-35, Jun. 2025.

[58] Cybersecurity and Infrastructure Security Agency, "Zero trust maturity model," CISA, Versão 2.0, Apr. 2023.

[59] J. Kindervag, "No more chewy centers: Introducing the zero trust model of information security," Forrester Research, Tech. Rep., 2010.

[60] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, "A survey on zero trust architecture: Challenges and future trends," *Wireless Communications and Mobile Computing*, vol. 2022, p. 6476274, 2022.

[61] B. A. Ali, S. Hijjawi, L. Campbell, M. Gregory, and S. Li, "A maturity framework for zero-trust security in multiaccess edge computing," *Security and Communication Networks*, vol. 2022, p. 3178760, 2022.

[62] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.

[63] S. Li, M. Iqbal, and N. Saxena, "Future industry internet of things with zero-trust security," *Information Systems Frontiers*, vol. 26, no. 5, pp. 1653–1666, 2024.

[64] R. Vanickis, P. Jacob, S. Dehghanzadeh, and B. Lee, "Access control policy enforcement for zero-trust-networking," in *Proc. 29th Irish Signals and Systems Conference (ISSC)*, 2018.

[65] Z. Zaheer, H. Chang, S. Mukherjee, and J. Van der Merwe, "eZTrust: Network-independent zero-trust perimeterization for microservices," in *Proc. ACM Symposium on SDN Research (SOSR)*, 2019, pp. 49–61.

[66] E. Lear, R. Droms, and D. Romascanu, "Manufacturer usage description specification," RFC 8520, IETF, 2019.

[67] D. Dodson, D. Montgomery, T. Polk, M. Ranganathan, M. Souppaya *et al.*, "Securing small-business and home internet of things (IoT) devices: Mitigating network-based attacks using manufacturer usage description (MUD)," National Institute of Standards and Technology, NIST Special Publication 1800-15, 2021.